

Data-efficient deep operator network for unsteady flow: A multi-fidelity approach with physics-guided subsampling

Sunwoong Yang ^a, Youngkyu Lee ^b, Namwoo Kang ^{a,c,*}

^a Cho Chun Shik Graduate School of Mobility, Korea Advanced Institute of Science and Technology (KAIST), Daejeon, 34051, Republic of Korea

^b Division of Applied Mathematics, Brown University, 182 George St, Providence, 02906, RI, USA

^c Narnia Labs, Daejeon, 34051, Republic of Korea

ARTICLE INFO

Keywords:

Deep operator networks
Multi-fidelity modeling
Transfer learning
Physics-guided subsampling
Small dataset
Spatio-temporal flow prediction

ABSTRACT

This study presents an enhanced multi-fidelity deep operator network (DeepONet) framework for efficient spatio-temporal flow field prediction, with particular emphasis on practical scenarios where high-fidelity data is scarce. We introduce several key innovations to improve the framework's efficiency and accuracy. First, we enhance the DeepONet architecture by incorporating a merge network that enables more complex feature interactions between operator and spatio-temporal coordinates, achieving a 50.4 % reduction in prediction error compared to traditional dot-product operations. We further optimize the architecture through temporal positional encoding and point-based subsampling strategies, achieving a 7.57 % improvement in prediction accuracy while reducing training time by 96 %. Building upon this foundation, we develop a transfer learning-based multi-fidelity framework that leverages knowledge from pre-trained low-fidelity models to guide high-fidelity predictions. Our approach freezes the pre-trained branch and trunk networks while making only the merge network trainable during high-fidelity training, preserving valuable low-fidelity representations while efficiently adapting to high-fidelity features. Through systematic investigation, we demonstrate that this fine-tuning strategy not only outperforms linear probing and full-tuning alternatives but also surpasses conventional multi-fidelity frameworks by up to 76 %, while achieving up to 43.7 % improvement in prediction accuracy compared to single-fidelity training. As another core contribution, we introduce a physics-guided subsampling approach, which strategically selects high-fidelity training spatial points based on temporal dynamics identified by pre-trained low-fidelity models. This physics-guided subsampling strategy demonstrates remarkable effectiveness, achieving prediction accuracy comparable to a baseline model while reducing the high-fidelity sample requirement by 40 %. The robustness of these contributions, particularly our proposed MF-DeepONet framework and time-derivative subsampling, is further validated on a second, more complex dataset where they continue to demonstrate superior or highly competitive performance against conventional benchmarks. Through comprehensive experiments across multiple resolutions and two distinct datasets of increasing complexity, our enhanced framework demonstrates the potential to reduce the required high-fidelity dataset size while maintaining predictive accuracy.

* Corresponding author.

E-mail addresses: sunwoongy@kaist.ac.kr (S. Yang), youngkyu_lee@brown.edu (Y. Lee), nwkang@kaist.ac.kr (N. Kang).

URL: <https://sites.google.com/view/aerodat> (S. Yang), <https://sites.google.com/view/youngkyulee> (Y. Lee), <https://www.smartdesign-lab.org/people/professor> (N. Kang)

<https://doi.org/10.1016/j.cma.2025.118254>

Received 23 March 2025; Received in revised form 17 July 2025; Accepted 18 July 2025

Available online 9 August 2025

0045-7825/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

1. Introduction

In the era of scientific machine learning (SciML), accurate and efficient prediction of spatio-temporal flow fields plays a crucial role across various engineering disciplines, particularly in the development of digital twins that require real-time flow field predictions for virtual replicas of physical systems. While traditional numerical approaches like computational fluid dynamics (CFD) offer high accuracy, their computational demands make them impractical for real-time applications, where rapid prediction and flexible adaptation to different flow parameters are essential. This has led to the emergence of various deep learning approaches, including multilayer perceptrons (MLPs) [1,2], convolutional neural networks (CNNs) [3–8], and graph neural networks (GNNs) [9–12], as surrogate models for flow field prediction. However, these architectures face inherent limitations: MLPs lack spatial structure awareness and cannot flexibly handle varying mesh resolutions, requiring retraining for each new mesh configuration—a significant drawback for digital twins that must adapt to different system configurations. CNNs are constrained by fixed regular grid resolutions, limiting their applicability to irregular domains. GNNs, while capable of handling unstructured meshes, require extensive memory resources to store and process mesh connectivity information, making them computationally intensive for large-scale problems [13].

The deep operator network (DeepONet) [14], introduced to learn mappings between function spaces, offers a compelling alternative for spatio-temporal flow field prediction tasks by overcoming the limitations of traditional methods [15–19]. Its unique architecture, consisting of separate branch and trunk networks, enables prediction at arbitrary spatial locations without being constrained by specific grid resolutions or mesh connectivities. This resolution-invariant property makes DeepONet particularly suitable for flow field predictions where data may come from simulations with varying mesh configurations. However, the standard DeepONet architecture may not be ideally designed to capture the complex dynamics and intricate features of spatio-temporal behavior [16,20–22]. To fully exploit DeepONet's potential for spatio-temporal flow field prediction, we recognize the need for specialized architectural refinements and training methodologies tailored to fluid flow applications. By systematically developing and integrating enhancements to the core DeepONet framework, we aim to significantly improve both the accuracy and computational efficiency of flow field predictions through: 1) expanding the architecture with additional networks for more flexible feature interactions between operator and spatio-temporal coordinates, 2) addressing spectral bias through strategic positional encoding implementation, and 3) implementing efficient point-based subsampling techniques with automatic mixed precision training to reduce computational overhead.

However, training DeepONet remains challenging, especially when high-fidelity data is limited due to computational constraints. In this context, multi-fidelity modeling has emerged as a powerful approach in SciML, effectively leveraging relatively abundant low-fidelity data alongside limited high-fidelity data to enhance prediction accuracy while maintaining computational efficiency [23–28]. Existing multi-fidelity DeepONet frameworks [29–31] predominantly employ coupled architectures, where low-fidelity (LF) and high-fidelity (HF) networks are trained and utilized together. This coupling approach fundamentally requires identical query points for both branch inputs (function queries) and trunk inputs (spatio-temporal coordinates) across different fidelity levels. This severely restricts DeepONet's primary advantage: the flexibility to leverage operators at arbitrary points. It forces researchers to design low-fidelity and high-fidelity experiments with identical branch inputs and trunk inputs beforehand—a requirement that is highly inefficient and often impractical in industrial applications where data collection follows operational constraints rather than organized experimental design. While the multi-fidelity DeepONet proposed by Howard et al. [32] successfully addressed this constraint by adopting a multi-fidelity deep neural network approach [26,33], it still requires the coupled use of LF and HF networks during inference. This coupled architecture introduces computational overhead by requiring sequential two-network inference, which increases both memory requirements and computational costs. The inefficiency scales poorly with multiple fidelity levels, as each additional fidelity layer requires its own network, creating cascading overhead particularly problematic when integrated with physics-informed methods, where automatic differentiation should propagate through all architectures.

Due to this inherent coupling and the availability of matching LF and HF datasets, these approaches naturally adopt residual learning architectures. In these frameworks, LF DeepONet is trained specifically on LF data to produce baseline predictions, while a second separate network, residual DeepONet, is trained to model the discrepancy between the low-fidelity and high-fidelity outputs (i.e., $y_{HF} = y_{LF} + \hat{y}_{residual}$). More fundamentally, residual approaches assume that low-fidelity simulations capture essential physical phenomena with less accuracy—an assumption that can be challenging in fluid dynamics, where low-resolution simulations may struggle to fully represent critical structures like vortex formations or boundary layer effects. When LF and HF data differ in complex ways, residual DeepONet model must focus on modeling these complicated residual patterns, which is highly inefficient compared to directly predicting the HF values. This inefficiency stems from forcing the network to learn fundamentally missing physics rather than leveraging the strengths of end-to-end learning. The residual network must not only correct numerical inaccuracies but also compensate for entirely absent physical phenomena in the baseline LF DeepONet model, essentially requiring it to learn more complex mappings than a direct prediction approach would need.

Beyond these architectural limitations, fundamental methodological challenges exist in effectively leveraging the relationship between LF and HF data during DeepONet training. While Xu et al. [34] proposed a two-phase approach pre-training an LF DeepONet before training a residual DeepONet—which removes the requirement of identical LF and HF datasets by using its sequential transfer learning approach—their framework still necessitates two separate networks during inference stage. This decoupling between LF and HF DeepONets during training phase allows for more flexibility in dataset design, but continues to suffer from computational inefficiency as both models must be employed during inference. Additionally, it lacks a systematic mechanism to identify critical regions where high-fidelity training would be most beneficial, resulting in potentially inefficient use of the pre-trained low-fidelity model and valuable high-fidelity data. Combined with the inherent drawbacks of residual-based architectures discussed earlier, these issues highlight the need for a fundamentally different approach to multi-fidelity operator learning.

A more refined approach to multi-fidelity DeepONet for spatio-temporal flow field prediction would incorporate four key innovations: (1) a unified network structure that eliminates the need for coupled use of low-fidelity and high-fidelity DeepONets during inference, reducing computational overhead and memory requirements; (2) preservation of DeepONet's inherent query flexibility by removing the requirement for low-fidelity and high-fidelity datasets to share identical function inputs or spatio-temporal query points; (3) a physics-guided subsampling strategy that identifies regions of significant flow dynamics using pre-trained low-fidelity models, strategically focusing the training process on high-fidelity data from these dynamically critical areas to maximize information gain while minimizing computational cost; and (4) comprehensive evaluation of the effects of dataset fidelities and sizes on multi-fidelity model performance, with particular emphasis on strategies for minimizing high-fidelity data requirements while maintaining prediction accuracy.

To address these challenges comprehensively, we present a systematic investigation that directly tackles all four aspects through enhanced DeepONet architectures and efficient training strategies for spatio-temporal flow prediction, with a particular focus on developing practical multi-fidelity frameworks. Our key contributions can be organized into two main thrusts.

Our first step involves designing targeted enhancements to the DeepONet architecture for more accurate spatio-temporal flow field predictions:

1. We propose an expanded DeepONet architecture incorporating a merge network for complex feature interactions between operator (branch net) and spatio-temporal coordinate spaces (trunk net). Through extensive experiments, we demonstrate that our element-wise multiplication merge strategy achieves a 50.4 % reduction in prediction error compared to traditional dot-product operations, while maintaining similar model parameters. This significant improvement highlights the importance of non-linear feature processing in capturing intricate flow dynamics.
2. We systematically investigate positional encoding to establish practical guidelines for its application to spatio-temporal DeepONets. Our analysis reveals a crucial insight for physical systems: contrary to common practice in computer vision domains, encoding spatial coordinates degrades performance, while selective temporal encoding is beneficial, yielding a 7.57 % improvement in prediction accuracy. This provides a valuable, non-obvious guideline for optimizing coordinate representation in spatio-temporal operator learning.
3. We leverage DeepONet's unique point-based prediction capability to develop efficient training protocols. Our approach demonstrates that training with a subset of spatial points significantly reduces computational overhead while maintaining prediction accuracy—challenging the assumption that more training points can lead to better results. Combined with automatic mixed precision training, we achieve a remarkable 96 % reduction in training time (from 8.672 to 0.350 h) while maintaining or even enhancing prediction accuracy.

Second, we develop and validate a novel multi-fidelity DeepONet (MF-DeepONet) framework that efficiently leverages both low- and high-fidelity data:

1. We introduce a transfer learning-based MF-DeepONet framework that fundamentally departs from sequential, two-network approaches like those in [29–31]. By strategically transferring knowledge from pre-trained low-fidelity models to a single unified network for high-fidelity predictions, our approach eliminates both the computational overhead of maintaining coupled networks and the restrictive constraint of matching query points between fidelity levels: where the transfer learning strategy is inspired by previous works that have successfully applied transfer learning techniques to multi-fidelity modeling [35–42]. This critical advancement preserves DeepONet's fundamental capability to evaluate operators at arbitrary points while removing the need for aligned branch and trunk inputs across fidelity levels—enabling seamless integration of heterogeneous data from different sources and resolutions. In direct comparison with conventional multi-fidelity approaches proposed by Lu et al. [29], our framework demonstrates superior performance with up to 76 % improvement in prediction accuracy while significantly reducing computational requirements during both training and inference, and the robustness of such performance is further validated on a second, more challenging high-resolution dataset.
2. Through systematic investigation of different transfer learning configurations—including fine-tuning (where only the merge network is trainable while branch and trunk networks are frozen), full-tuning (where all network components are retrained), and linear probing (where only the final layer of the merge network is trainable)—we demonstrate that the selective fine-tuning approach significantly outperforms alternatives. Our extensive experimentation with various low-fidelity resolutions (16×16 , 32×32 , 64×64) and dataset sizes (50–300 samples) reveals that fine-tuning achieves up to 43.7 % improvement in prediction accuracy compared to single-fidelity training, while maintaining computational efficiency.
3. We propose a physics-guided subsampling strategy based on temporal derivatives that efficiently identifies critical spatial areas requiring high-fidelity training. This intuitive approach demonstrates remarkable effectiveness: our method reduces prediction error by 10.02–20.73 % compared to traditional uniform sampling methods using the same amount of data. Furthermore, using only 60 % of high-fidelity samples with the same low-fidelity data, we achieve similar prediction accuracy compared to models trained without physics-guided sampling. The robustness of this physics-guided subsampling is also demonstrated on a more complex, higher-resolution dataset, where it remains an effective strategy that performs competitively against conventional residual-based sampling. These findings suggest a promising direction for reducing reliance on dense high-fidelity data in multi-fidelity modeling.

The remainder of this paper is organized as follows. [Section 2](#) presents the theoretical foundations of our enhanced DeepONet architecture and multi-fidelity framework, including the merge network design and time-derivative guided subsampling strategy. [Section 3](#) describes the flow field data generation process with varying initial conditions and fidelity levels. [Section 4](#) evaluates the

architectural enhancements to DeepONet, analyzing the effects of different merge processes, positional encoding, and efficient training techniques. [Section 5](#) investigates our MF-DeepONet framework, comparing different transfer learning approaches and demonstrating performance with limited high-fidelity data. [Section 6](#) explores the time-derivative guided subsampling strategy and quantifies its effectiveness in reducing high-fidelity data requirements. [Section 7](#) further validates our framework on a second, more complex dataset and provides a comparative analysis of different guided sampling strategies. Finally, [Section 8](#) summarizes our findings and discusses implications for future research.

2. Methodology

2.1. DeepONet architecture enhancement

2.1.1. Enhanced DeepONet with merge network

The original DeepONet consists of two main components: a branch network and a trunk network. The branch network maps the input function information (e.g., initial conditions, boundary conditions, or other function characterizations) to a latent representation $\mathbf{B}(\mathbf{u}) \in \mathbb{R}^p$, where $\mathbf{u}$ is input function and p is the dimension of the latent space. The trunk network takes spatio-temporal coordinates, $\mathbf{x}$; and it outputs latent space, $\mathbf{T}(\mathbf{x}) \in \mathbb{R}^p$. In the standard DeepONet formulation, these two latent representations are combined through a simple dot product operation to produce the final prediction: $f(\mathbf{u})(\mathbf{x}) = \mathbf{B}(\mathbf{u}) \cdot \mathbf{T}(\mathbf{x})$ where function f is an operator modeled by DeepONet.

For complex spatio-temporal flow field predictions, we hypothesize that this simple dot product operation may not provide sufficient non-linear modeling capacity. While Lu et al. [14] demonstrated that incorporating a bias term in the dot product slightly improves the model's performance in their applications, the effectiveness of this modification in complex flow field prediction remains to be verified. Therefore, we propose enhancing the architecture with a third component—a merge network—and investigate four strategies for merging branch and trunk network outputs, ranging from simple dot products to more complex non-linear operations through proposed merge networks:

- **Merge type 0 (without bias):** Simple dot product from the original DeepONet:

$$\text{Final output} = \mathbf{B}(\mathbf{u}) \cdot \mathbf{T}(\mathbf{x}) \quad (1)$$

- **Merge type 0 (with bias):** Dot product with learnable bias for enhanced flexibility [14]:

$$\text{Final output} = \mathbf{B}(\mathbf{u}) \cdot \mathbf{T}(\mathbf{x}) + b \quad (2)$$

- **Merge type 1:** Element-wise multiplication with additional non-linear processing by merge network:

$$\text{Final output} = \text{MergeNet}(\mathbf{B}(\mathbf{u}) \odot \mathbf{T}(\mathbf{x})) \quad (3)$$

- **Merge type 2:** Concatenation with additional non-linear processing by merge network:

$$\text{Final output} = \text{MergeNet}([\mathbf{B}(\mathbf{u}); \mathbf{T}(\mathbf{x})]) \quad (4)$$

The introduction of MergeNet in Types 1 and 2, as visually represented in [Fig. 1](#), is intended to address a critical limitation of the original DeepONet architecture by enabling more complex non-linear interactions between branch and trunk network outputs. While the simple dot product operation in Type 0 may suffice for simpler operator learning tasks, more sophisticated non-linear processing of branch and trunk outputs can enhance complex spatio-temporal flow field predictions. The merge network provides this crucial non-linear modeling capacity by further processing the combined features through additional neural network layers, enabling the model to capture intricate relationships and dynamic patterns that may be missed by the basic dot product operation alone.

2.1.2. Enhanced DeepONet with positional encoding

Positional encoding (PE) has been widely used in various deep learning architectures, particularly in transformers and attention-based models, to incorporate spatial information into the input representations [43,44]. In the context of DeepONet, the trunk net processes coordinate information which can suffer from spectral bias, limiting its ability to capture high-frequency components and intricate spatio-temporal dependencies [45,46]. By mapping coordinates to a higher-dimensional space beyond the original (x, y, t) representation, PE is expected to enhance the network's ability to learn complex flow dynamics and temporal dependencies critical for accurate spatio-temporal flow field prediction.

Specifically, when the PE transforms input coordinate t to a vector $\gamma(t)$ using sinusoidal functions,

$$\gamma(t) = [a_1 \cos(2\pi B_1^T t), a_1 \sin(2\pi B_1^T t), \dots, a_{m/2} \cos(2\pi B_{m/2}^T t), a_{m/2} \sin(2\pi B_{m/2}^T t)]^T \quad (5)$$

where $B_j \in \mathbb{R}$ are frequency vectors sampled from a normal distribution and scaled by a factor σ , a_j are amplitude factors (set to 1 in our implementation), and m is the mapping size determining the dimension of the encoded features. This transformation, visualized in [Fig. 2](#), maps the input to a higher-dimensional space, enabling better representation of high-frequency functions.

The PE implementation includes two key hyperparameters:

- **Scale factor (σ):** Controls the frequency range of the encoding, affecting the model's ability to capture different scales of variation.
- **Mapping size (m):** Determines the dimension of the encoded features, controlling the expressiveness of the encoding.

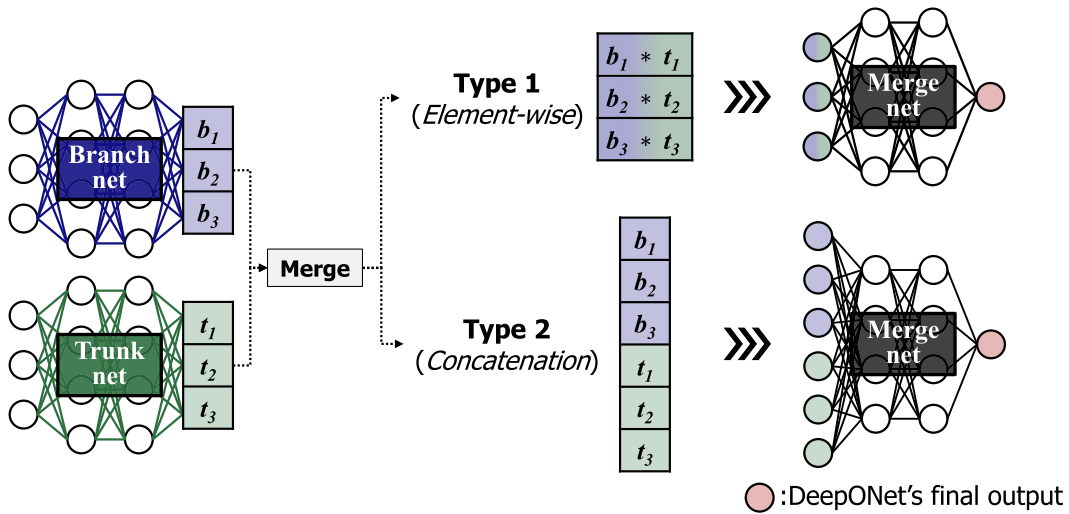


Fig. 1. Schematic of two types of merging processes proposed in this study: Type 1 (element-wise) and Type 2 (concatenation).

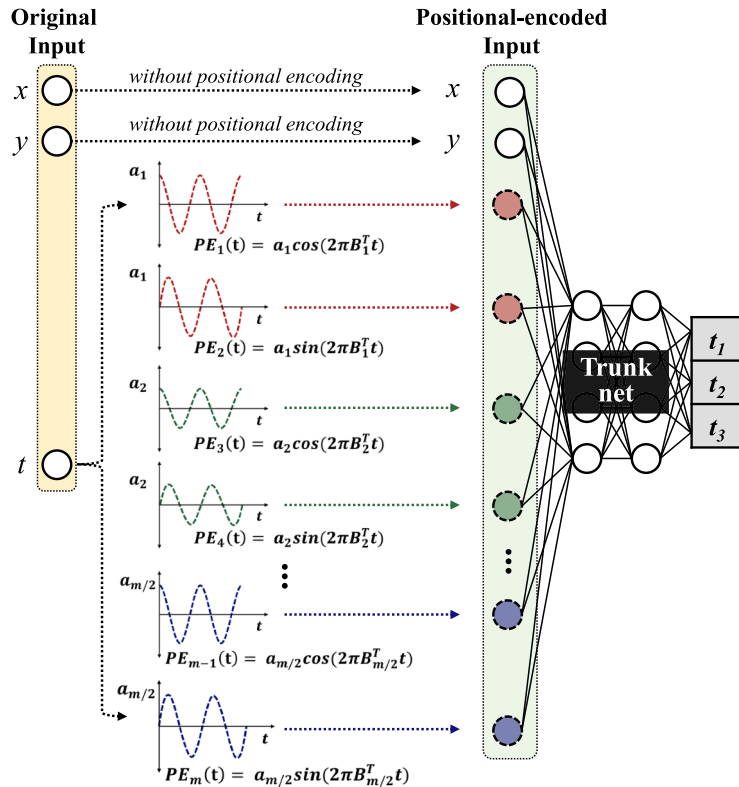


Fig. 2. Schematic of the positional encoding process: this example shows the process where only temporal coordinate (t) is encoded.

Additionally, we introduce a flexible option that determines the scope of encoding:

- **PE(t)**: PE is applied only to the temporal coordinate in the trunk network input, preserving the original spatial coordinates.
- **PE(x, y)**: PE is applied only to the spatial coordinates in the trunk network input, preserving the original temporal coordinate.
- **PE(x, y, t)**: PE is applied to all coordinates (spatial and temporal) in the trunk network input, providing a uniform encoding scheme.

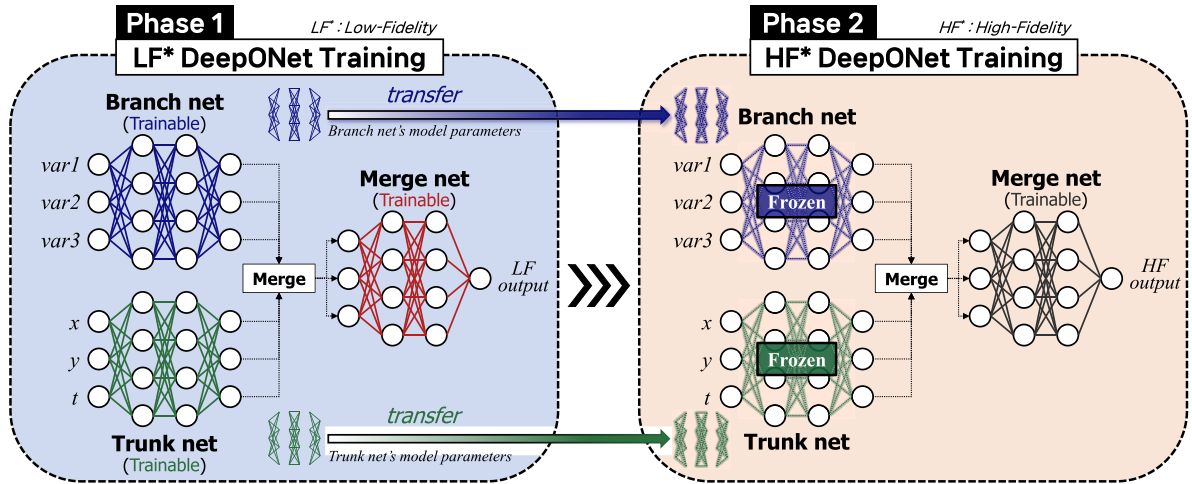


Fig. 3. Two-phase training process of the MF-DeepONet framework. In **Phase 1**, all components are trained on low-fidelity data. In **Phase 2**, all networks are transferred but branch and trunk networks are frozen, while only the merge network is fine-tuned with high-fidelity data.

2.2. Multi-fidelity DeepONet framework with physics-guided subsampling strategy

2.2.1. Multi-fidelity DeepONet framework

We propose a multi-fidelity DeepONet (MF-DeepONet) framework that efficiently leverages both low-fidelity and high-fidelity data through knowledge transfer. Our two-phase approach first trains an LF DeepONet on relatively abundant low-fidelity data, then strategically transfers these pre-trained networks (LF DeepONet) to the high-fidelity model (HF DeepONet). By freezing the branch and trunk parameters of HF DeepONet (see Fig. 3), we preserve the fundamental flow physics captured during low-fidelity training. During the high-fidelity training phase, only the merge network remains trainable, allowing it to learn the specific refinements necessary for accurate high-fidelity predictions. Importantly, the merge network parameters transferred from Phase 1 serve as initial values that are further optimized using the high-fidelity dataset.

A key advantage of our approach is its computational efficiency during inference. While training involves two distinct phases, once training is complete, only the final HF DeepONet model in Phase 2 is needed for predictions. This creates a significant computational advantage compared to conventional coupled multi-fidelity frameworks that require both low and high-fidelity networks to be executed in sequence during inference. Our decoupled design effectively eliminates this computational overhead, resulting in substantially faster inference times and reduced memory requirements during deployment—critical factors for real-time applications and resource-constrained environments. Furthermore, this decoupled architecture enables the use of flexible LF and HF datasets, as it doesn't require corresponding data pairs between fidelity levels.

To comprehensively understand the impact of different knowledge transfer strategies, we will explore three distinct transfer learning approaches within our framework. While our primary proposed MF-DeepONet strategy employs fine-tuning (where only the merge network is trainable while branch and trunk networks are frozen), we additionally investigate [47]: (1) full-tuning, where all network components (branch, trunk, and merge) of HF DeepONet are trainable, allowing complete adaptation to high-fidelity data; and (2) linear probing, the most restrictive approach that freezes all layers except the final layer of the merge network.

2.2.2. Physics-guided subsampling strategy

After training the LF DeepONet model, we aim to efficiently utilize the available HF dataset for training the HF DeepONet. While DeepONet's point-based architecture already offers flexibility in training with either complete or subsampled spatial points (note that effects of subsampling will be investigated in Section 4.2), we propose further enhancing this capability through physics-guided subsampling. Specifically, we introduce a time-derivative subsampling technique that strategically identifies dynamically important regions in the flow field. This approach leverages the temporal derivative information from the pre-trained LF model to guide the selection of spatial query points from the HF dataset, potentially enabling more efficient training of HF DeepONet by focusing computational resources on regions with significant flow dynamics.

The complete procedure for selecting HF training points using time-derivative guided subsampling is formally described in Algorithm 1. This physics-guided approach first computes the temporal derivatives at each spatial point from the HF dataset using the pre-trained LF model through automatic differentiation (lines 4–6). For each HF spatial location, we calculate a score based on the average magnitude of temporal derivatives predicted by the LF model (lines 7–9). Points with larger temporal derivatives—indicating regions of significant flow dynamics—receive higher sampling probabilities after score transformation and normalization (lines 10–11). Rather than deterministically selecting the highest-scoring points, we employ probabilistic sampling using probability

Algorithm 1 Time-derivative guided subsampling strategy.

- 1: **Input:** HF dataset spatial coordinates $\mathbf{x}_{HF}$, pre-trained LF model, sampling ratio r
- 2: **Output:** Selected spatial points for HF DeepONet training
- 3: **Note:** $\hat{\omega}^{LF}(\mathbf{x}_{HF})$ denotes the vorticity field predicted by the LF model at coordinates $\mathbf{x}_{HF}$
- 4: **for** each spatial point $\mathbf{x}_{HF}$ **do**
- 5: Compute $\frac{\partial \hat{\omega}^{LF}(\mathbf{x}_{HF})}{\partial t}$ using automatic differentiation
- 6: **end for**
- 7: **for** each spatial point $\mathbf{x}_{HF}$ **do**
- 8: $\text{score}(\mathbf{x}_{HF}) = \frac{1}{T} \sum_{t=1}^T \left| \frac{\partial \hat{\omega}^{LF}(\mathbf{x}_{HF})}{\partial t} \right|$
- 9: **end for**
- 10: Transform scores using power-law: $p(\mathbf{x}_{HF}) = \text{score}^2(\mathbf{x}_{HF})$
- 11: Normalize: $p(\mathbf{x}_{HF}) = \frac{p(\mathbf{x}_{HF})}{\sum p(\mathbf{x}_{HF})}$
- 12: To subsample $N_{\text{subsampled}}$ spatial points from the HF dataset for training HF DeepONet:
- 13: Sample $N_{\text{subsampled}} \cdot r$ points using multinomial distribution with probabilities $p(\mathbf{x}_{HF})$
- 14: Sample remaining $N_{\text{subsampled}} \cdot (1 - r)$ points uniformly at random

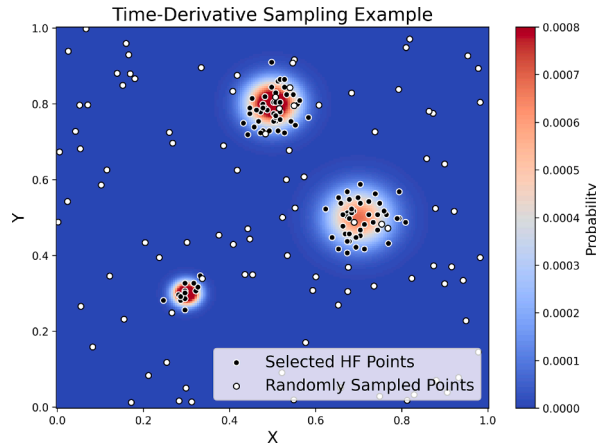


Fig. 4. Time-derivative guided subsampling demonstration: background shows probability distribution based on temporal derivatives, with black dots representing points selected using this distribution and white circles showing randomly sampled points for spatial coverage.

defined by these score values, which introduces beneficial stochasticity while still favoring dynamically important regions. The final training set of $N_{\text{subsampled}}$ points is constructed by first selecting a fraction ($N_{\text{subsampled}} \cdot r$) via the probabilistic approach (line 13), and then sampling the remaining points ($N_{\text{subsampled}} \cdot (1 - r)$) uniformly to ensure spatial coverage (line 14). By strategically focusing computational resources on dynamically important regions while preserving randomness, this approach can achieve superior prediction accuracy by using a more informative, smaller subset of HF spatial training points than a purely random sampling would. Fig. 4 visually demonstrates our time-derivative guided subsampling strategy, showing how high-fidelity training points are strategically selected based on a combination of derivative-weighted probability distributions and uniform random sampling to maintain spatial coverage.

2.2.3. Key considerations for effective multi-fidelity modeling

Our MF-DeepONet framework with physics-guided subsampling fundamentally relies on the predictive capability of the pre-trained LF DeepONet, meaning its effectiveness is inherently tied to the quality and computational efficiency of the low-fidelity dataset. Therefore, a critical preliminary step in applying any MF strategy, including ours, is the careful selection and validation of the LF model. This selection process must consider two key aspects: (1) the degree to which the LF model can capture the essential physical phenomena relevant to the HF model, as this dictates the potential for effective knowledge transfer, and (2) the computational cost of generating LF data, since an MF approach achieves its overall efficiency gain primarily when LF simulations are meaningfully less computationally expensive than HF simulations. If the LF data are not significantly less expensive to generate, or if they fail to adequately represent the critical physics of the HF data, the advantages of the multi-fidelity approach are naturally diminished. However, this limitation is not unique to our framework—it represents a common challenge shared across multi-fidelity modeling strategies that incorporate LF data, whether explicitly or implicitly.

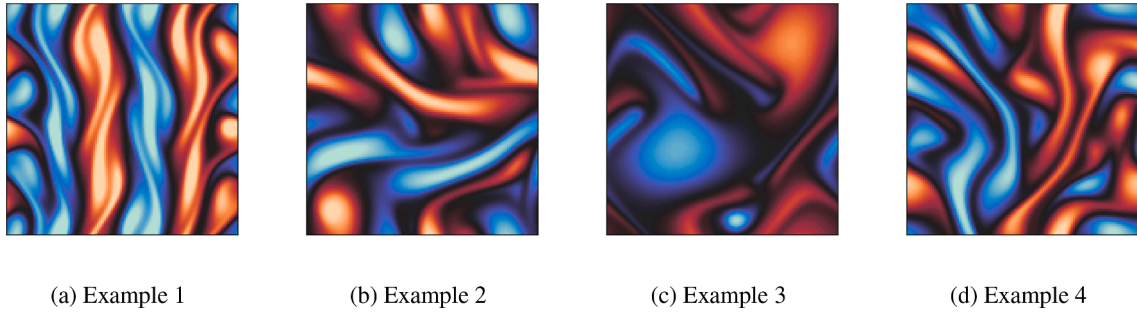


Fig. 5. Vorticity fields at $t = 0.5s$ for four distinct initial conditions, each defined by a different combination of var_1 and var_2 . This diversity in the observed patterns highlights the complexity and challenges of predicting the flow evolution.

3. Data generation: Flow fields with varying initial conditions

The training datasets comprise both LF and HF flow simulations of incompressible Navier-Stokes equations in a two-dimensional periodic domain $[0, 1m] \times [0, 1m]$. The numerical solution is obtained using a pseudo-spectral solver that employs fast Fourier transforms for spatial discretization, with a 2/3 dealiasing rule to prevent aliasing errors. The solver implements a semi-implicit time integration scheme where advection terms are treated explicitly while diffusion terms are handled implicitly.

First, for the HF dataset, we generate 100 training samples and 50 test samples on a uniform grid of 128×128 resolution. Each sample is initialized with different combinations of two parameters (var_1, var_2) that define the initial velocity field components:

$$\begin{aligned} v_x(x, y, t = 0) &= -\sin(var_1 \pi y) \\ v_y(x, y, t = 0) &= \sin(var_2 \pi x) \end{aligned} \quad (6)$$

where var_1 and var_2 are randomly sampled from uniform distributions in the range $\mathcal{U}[1, 3]$. The Reynolds number is fixed at $Re = 1,000$ for all simulations. The temporal evolution of the flow field is computed with a time step of $\Delta t = 2.5 \times 10^{-5}[s]$, solving until $t = 1s$. While the solver operates at this fine temporal resolution, the flow field data is extracted at a coarser interval of $\Delta t_{train} = 5 \times 10^{-3}[s]$ for training purposes. This output frequency is selected as it adequately captures relevant flow dynamics while significantly reducing the number of temporal snapshots. This approach enhances computational efficiency for the subsequent AI model training, which can effectively learn the continuous dynamics from this sparser data, thus reducing overall training overhead. Importantly, these simulations feature no external forcing after $t = 0$, resulting in gradually dissipating flow fields where vorticity magnitude decreases over time. This dissipative behavior creates challenging temporal dynamics that are particularly difficult to predict accurately as the flow evolves further from its initial state. Throughout this study, we focus on predicting the vorticity field, a derived quantity that captures the local rotation in the fluid and presents a more challenging prediction target than primary variables like velocity components. Fig. 5 displays the vorticity field at $t = 0.5s$ for four distinct initial conditions. The marked diversity in the flow patterns, resulting from variations in var_1 and var_2 , underscores the complex nature of the dataset.

For the LF datasets, we generate 300 training samples for each of three different grid resolutions: 16×16 , 32×32 , and 64×64 . While the generation of HF 128×128 data requires approximately 360 s per sample using AMD EPYC 7282 processor, the LF data require 90, 100, and 140 s for 16×16 , 32×32 , and 64×64 resolutions, respectively. In total, generating the HF dataset (100 training samples) requires 10 h, whereas generating each LF dataset (300 samples) requires 7.5, 8.3, and 11.7 h, respectively. As illustrated in Fig. 6, the grid resolution significantly impacts the accuracy of flow field predictions. The coarsest resolution (16×16) shows severe limitations in capturing flow evolution: while it approximates the macroscopic flow patterns at $t = 0.25s$, the flow field becomes notably distorted from $t = 0.5s$ onward compared to higher-resolution simulations. This suggests that data from such coarse resolutions might be inadequate or even detrimental in a multi-fidelity framework. However, from 32×32 resolution upward, the simulations begin to capture the essential macroscopic flow properties visible in the HF (128×128) case. This observation suggests that LF data from these intermediate resolutions (32×32 and 64×64) could effectively support HF predictions in our multi-fidelity framework.

Note on dataset nesting in multi-fidelity modeling: Although the LF datasets used in this study are spatially nested within the HF datasets—i.e., their grid points are subsets of the higher-resolution grids (e.g., 64×64 is fully contained within 128×128)—the input function parameters (var_1 and var_2) are sampled independently across fidelities. Consequently, the datasets are not nested in terms of input function parameters. This is a critical distinction because multi-fidelity strategies often assume corresponding LF and HF outputs exist for identical input queries [29–31], which is not the case here due to independent parameter sampling across fidelities. This design choice aligns with our objective to develop an MF-DeepONet framework that operates robustly without requiring paired or nested datasets, thereby enhancing its practical applicability in real-world scenarios where consistent parameter sampling across fidelities is often infeasible. It should also be noted that our framework can accommodate datasets with non-nested spatio-temporal grid points, which is not explored in this study.

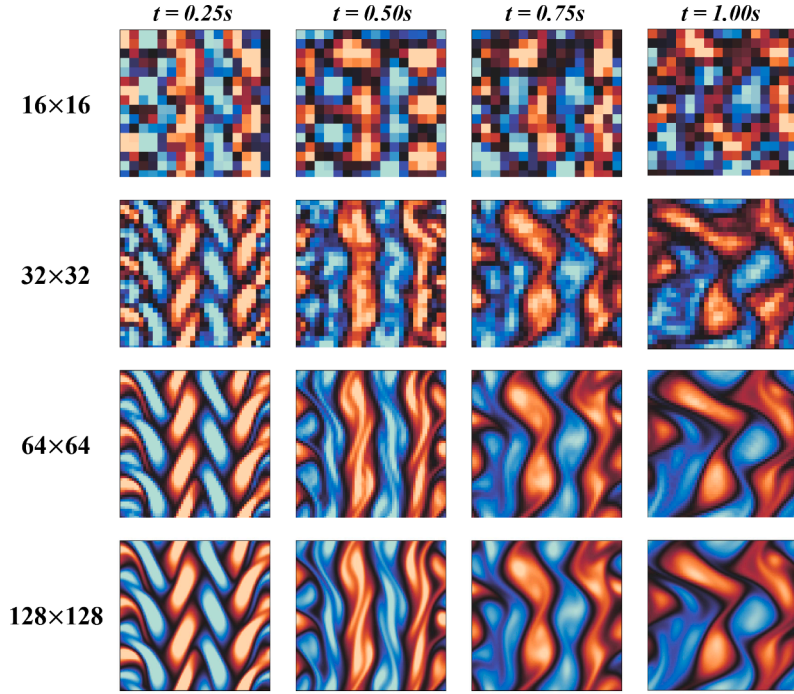


Fig. 6. Effects of grid resolution on vorticity field evolution with fixed parameters $var_1 = 3$ and $var_2 = 1$ (contours represent vorticity values). It also shows how dissipative effects intensify over time in the adopted flow field dataset.

4. Improvement of DeepONet architecture for spatio-temporal flow field prediction

In this section, we focus on enhancing single-fidelity DeepONet performance using only high-fidelity data (128×128 resolution). We systematically investigate the effects of different merge processes, spatial point subsampling approaches, and positional encoding strategies to establish the most effective base architecture. This optimized DeepONet architecture will serve as the foundation for both LF and HF models in our multi-fidelity framework presented later.

4.1. Effects of different merge process of branch/trunk nets

We first investigate the effectiveness of different strategies for combining outputs from the branch and trunk networks elaborated in Section 2.1.1. To ensure a fair comparison, we carefully adjust the network architectures to maintain similar numbers of learnable parameters across different configurations (approximately 300K parameters). For the basic dot-product variants, Merge Type 0 with and without bias, we employ more complex branch and trunk networks to match the parameter count of models with Type 1 and 2 networks. Additionally, we implement the modified DeepONet architecture proposed by Wang et al. [48], which introduces encoder layers for enhanced nonlinear interactions but still relies on the fundamental dot-product mechanism between branch and trunk networks at its core. Across all DeepONet configurations, the branch network consistently takes two input parameters (var_1 and var_2) defining initial conditions, while the trunk network processes three spatial-temporal coordinates (x , y , and t). For detailed network architectures, please refer to Table 1. All models are trained for 800 epochs using the Adam optimizer with an initial learning rate of 10^{-3} , GeLU activation functions, and minibatch size of 1 on an NVIDIA A6000 GPU. To account for training stochasticity and ensure reproducible results, all experiments in this paper are repeated at least three times with different random seeds, and we report the averaged values of these runs. To evaluate the predictive accuracy of each model, we report the test error as the root mean squared error (RMSE) computed across all spatio-temporal test points and test cases. Specifically, for a set of N_{test} test cases, each with N_t temporal snapshots and N_x spatial points, the RMSE is defined as:

$$\text{RMSE} = \sqrt{\frac{1}{N_{\text{test}} \cdot N_t \cdot N_x} \sum_{i=1}^{N_{\text{test}}} \sum_{j=1}^{N_t} \sum_{k=1}^{N_x} (\hat{\omega}_{ijk} - \omega_{ijk})^2} \quad (7)$$

where $\hat{\omega}_{ijk}$ and ω_{ijk} denote the predicted and ground truth vorticity values, respectively, for the i -th test case, at time t_j , and spatial location $\mathbf{x}_k$. All test errors reported in this paper, including those in Table 1, are expressed as these RMSE values; this definition is expected to ensure that the reported test error reflects a comprehensive average across all spatio-temporal predictions in the test set, providing a physically meaningful and consistent measure of overall model accuracy.

Table 1
Comparison of different merge strategies for DeepONet.

Merge Strategy	Network Architecture	Parameters	Training Hour	Test Error
Type 0 (Dot-Product w/o bias)	Branch: $2-128 \times 6$ Trunk: $3-64-128 \times 6$	306,304	6.197	34.447
Type 0 (Dot-Product w/ bias)	Branch: $2-128 \times 6$ Trunk: $3-64-128 \times 6$	306,305	6.453	34.546
Type 1 (Element-wise)	Branch: $2-128$ Trunk: $3-64-128$ Merge: $128 \times 6-64-32-16-1$	300,289	8.672	17.079
Type 2 (Concatenation)	Branch: $2-128$ Trunk: $3-64-128$ Merge: $256-128 \times 5-64-32-16-1$	292,033	8.399	23.060
Wang et al. [48]	Branch: $2-128$ Trunk: $3-64-128$ Merge: $256-128 \times 5-64-32-16-1$	302,336	17.918	32.099

The results in Table 1 demonstrate three key findings. First, incorporating merge networks significantly improves prediction accuracy compared to the two Type 0 approaches suggested by Lu et al. [14]. While the original dot-product architecture was successful for simpler operator learning tasks in other studies, our results show its limitations in complex flow field predictions: the test error with dot-product operations (both with and without bias) remains above 34, indicating substantial prediction inaccuracies. In contrast, the introduction of merge networks dramatically reduces the test error to 17–23, despite requiring longer training times (approximately 8.5 h compared to 6.2 h) due to the additional processing of branch and trunk outputs through the distinct merge network. This stark improvement in accuracy—reducing test error from 34.447 to 17.079, a 50.4 % improvement—highlights that additional non-linear processing of combined features is crucial for capturing complex spatio-temporal flow dynamics. The merge network effectively acts as a learnable non-linear mapping that can adapt to intricate relationships between the latent representations from branch and trunk networks, a capability that simple dot-product operations alone cannot provide.

Second, the comparison with Wang et al. [48]’s approach provides additional evidence for the limitations of dot-product mechanisms in complex flow field predictions. Despite having a similar parameter count (302,336 parameters), Wang et al. [48]’s method achieves a test error of 32.099, which is substantially higher than our Type 1 (17.079) and Type 2 (23.060) approaches. Moreover, this approach requires significantly longer training time (17.918 h) compared to our Type 1 (8.672 h) and Type 2 (8.399 h) methods, indicating computational inefficiency due to its additional architectural complexity. Notably, while this approach outperforms the basic dot-product operations (Type 0), it still falls short of our proposed merge network strategies in both accuracy and computational efficiency.

Third, among the enhanced architectures, Type 1 (element-wise multiplication) outperforms Type 2 (concatenation), achieving a test error of 17.079 compared to 23.060. This advantage stems from the structured interaction in Type 1, which more naturally aligns with well-established techniques such as modal decomposition (e.g., proper orthogonal decomposition) and spectral expansions, both of which have demonstrated strong performance in CFD.

- Proper orthogonal decomposition (POD) perspective:** Element-wise multiplication in Type 1 architectures directly mirrors the fundamental structure of POD, where complex flow fields are reconstructed through coefficients multiplied by basis modes [49]. In DeepONet, this correspondence is explicit: the branch network generates coefficients, while the trunk network produces basis functions—creating a natural factorization that resembles POD’s coefficient-times-basis-function formulation. This structural alignment with POD, which has proven highly effective in CFD for reduced-order modeling and flow reconstruction [3,8], explains why Type1’s element-wise multiplication outperforms Type2’s less structured concatenation approach.
- Spectral methods perspective:** Type 1 approach also aligns with spectral methods, where solutions are represented as weighted sums of basis functions [22]. Here, the trunk network acts as a global basis generator, while the branch network provides coefficients—akin to spectral expansions in Fourier or Laplace methods. Since spectral methods have already demonstrated strong performance in solving the Navier-Stokes equations [50,51], enforcing this structured decomposition allows Type 1 to better capture the underlying physics of flow evolution, outperforming the less structured feature fusion in Type 2.

To better understand the qualitative differences between these merge strategies, we visualize the predicted vorticity profiles along a vertical line at $x = 0.5m$ for different time steps in Fig. 7. These line plots show how vorticity values vary along the y-direction from 0 to $1m$, providing insight into each model’s ability to capture flow field features at different spatial locations. The visualization clearly demonstrates the limitations of the dot-product operation and the advantages of incorporating merge networks. At both time steps (Fig. 7a and b), the dot-product approach (without bias, which outperformed the with-bias variant) exhibits significant difficulties capturing local flow features, producing overly smoothed predictions that miss important vorticity fluctuations across the domain. In contrast, both Type 1 and Type 2 merge networks successfully reproduce the complex flow patterns, following the ground truth’s local peaks and valleys with notably higher fidelity.

Note on overfitting issues in over-parameterized DeepONet: To obtain the architectures summarized in Table 1, we conducted manual hyperparameter tuning across a wide range of model depths, hidden layer sizes, and merge configurations with varying

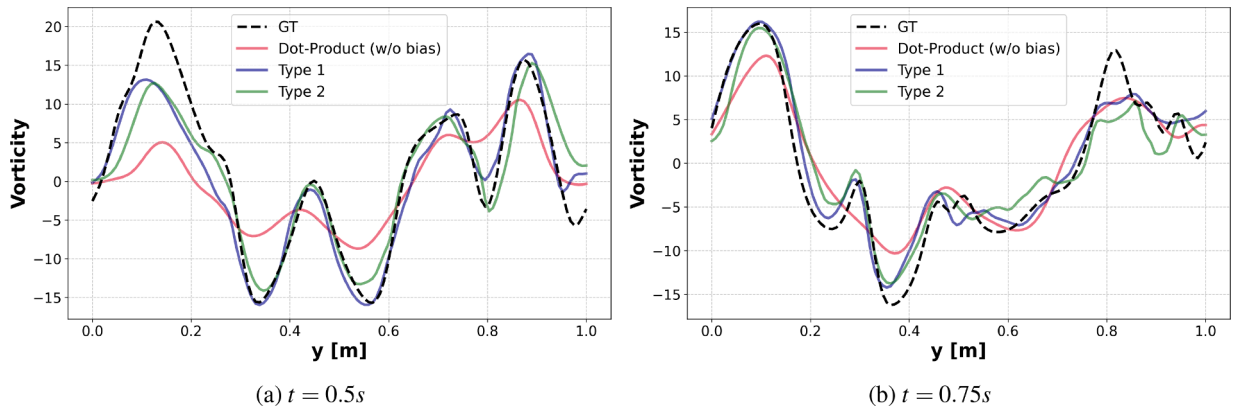


Fig. 7. Comparison of vorticity predictions along $x = 0.5m$ between different merge strategies. The plots show vorticity values versus y -coordinate for two different time steps, comparing predictions from dot-product operation (without bias) and merge network approaches (Type 1 and 2) against ground truth (GT). Both merge network types demonstrate superior ability in capturing local flow features and fluctuations compared to the basic dot-product operation.

parameter numbers. Despite testing deeper and wider networks with significantly larger parameter counts, we consistently observed that such models led to worse generalization performance—even as they achieved lower training losses. This suggests a critical insight: DeepONet tends to overfit easily when over-parameterized, indicating that its expressive power must be tightly controlled. The final architectures, each with approximately 300k parameters, were selected based on their superior balance between training efficiency and test accuracy.

4.2. Effects of spatial point subsampling and application of automatic mixed precision

Building upon the optimal merge network configuration (Type 1) identified in Section 4.1, we investigate the impact of spatial point subsampling and automatic mixed precision (AMP) training on model performance. Spatial point subsampling refers to our approach of selecting only a subset of available spatial locations from the full 128×128 grid for training, leveraging DeepONet's inherent capability to operate on arbitrary spatial points without requiring fixed grid structures. We explore three subsampling strategies: 16×16 , 32×32 , and 64×64 randomly selected points from the original 128×128 grid. The optimal choice of this subsampling ratio (e.g., the number of points like 16×16 or 32×32) can be highly problem-dependent, influenced by factors such as the physical characteristics of the investigated flow field, the scale of its primary dynamics, and overall data complexity. Therefore, for new applications, we recommend conducting a preliminary parametric study, similar to the exploration presented in Table 2, to determine a suitable subsampling level that effectively balances predictive accuracy with computational training efficiency.

AMP training refers to the strategic use of different numerical precisions (e.g., float32, float16) during model training to reduce memory usage and computational overhead without compromising model accuracy—a particularly relevant approach for DeepONet due to its intensive computational requirements in processing high-dimensional flow field data (for the details of AMP, please refer to Appendix A). For each subsampling configuration, we conduct experiments both with and without AMP training to systematically evaluate their combined effects on computational efficiency and prediction accuracy. All other training settings remain consistent with Section 4.1.

The results shown in Table 2 reveal important insights about DeepONet's training efficiency through the combination of sampling strategies and AMP training. First, examining the impact of spatial point sampling, we observe that smaller sampling sizes can maintain prediction accuracy while significantly reducing computational costs. With AMP enabled, the 16×16 sampling strategy

Table 2
Comparison of different spatial subsampling strategies with and without AMP training: Type 1 merge network is applied.

AMP	Number of Sampled Points	Training Hour	Test Error
ON	16×16	0.350	17.419
	32×32	0.785	18.184
	64×64	2.211	17.438
	128×128	8.672	17.079
OFF	16×16	0.377	16.876
	32×32	1.038	16.989
	64×64	3.458	17.862
	128×128	13.648	18.001

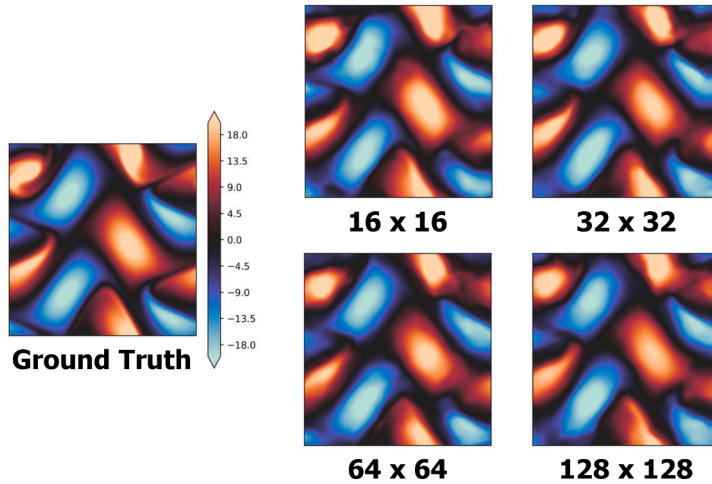


Fig. 8. To visually investigate the effects of number of subsampled points, 4 cases with AMP in Table 2 are visualized: snapshots at $t = 0.25s$. The comparison demonstrates that even with significantly reduced sampling (16×16), the model captures essential flow patterns similar to those observed in higher resolution samplings (32×32 , 64×64) and the full dataset (128×128).

achieves comparable accuracy (test error: 17.419) to the baseline that uses all available points (test error: 17.079), while dramatically reducing the training time from 8.672 to 0.350 h—a 96 % reduction. Similar trends are observed in cases without AMP, where the 16×16 sampling rather achieves a test error of 16.876 compared to 18.001 for full sampling due to its effective sampling nature compared to naive all-point-sampling approach.

The comparison between AMP and non-AMP training reveals interesting trade-offs in computational efficiency and prediction accuracy across different sampling sizes. While AMP-OFF training with 16×16 sampling achieves slightly better accuracy (16.876 versus 17.419 with AMP-ON) and requires slightly longer training time (0.377 versus 0.350 h), we opt for the AMP-ON configuration with 16×16 sampling in subsequent experiments. This choice is motivated by two practical considerations: (1) our comprehensive study involves numerous repeated experiments to account for training variability, where even small reductions in training time yield substantial cumulative savings; and (2) the computational advantage of AMP scales with sampling resolution—at full 128×128 sampling, AMP reduces training time by 36.5 %. This scalable efficiency demonstrates the practical value of AMP and underscores the importance of systematically exploring its effectiveness in various DeepONet settings to be investigated in later sections. Accordingly, the AMP-ON configuration with 16×16 sampling in Table 2 is judged to represent the reasonable compromise between accuracy and computational cost in this study.

In Fig. 8, we can observe the effects of different subsampling resolutions on flow field prediction quality given AMP. The comparison demonstrates that even with the coarsest sampling resolution (16×16), the DeepONet successfully captures the essential vorticity patterns and intensity distributions present in the flow field at $t = 0.25s$. As expected, the visual differences between sampling resolutions are minimal, with all configurations reproducing the key flow structures observed in the ground truth. This visual evidence supports our quantitative findings in Table 2, which indicate that the 16×16 sampling strategy achieves comparable prediction accuracy (MSE of 17.419) to the full 128×128 sampling (MSE of 17.079), while dramatically reducing the training time from 8.672 to 0.350 h. The consistent visual quality across different sampling resolutions highlights DeepONet’s point-based prediction flexibility, which allows for efficient data utilization without substantial degradation in prediction performance.

The findings in this section highlight a fundamental advantage of DeepONet over grid-based architectures like CNNs or GNNs: its point-based prediction capability enables remarkable flexibility in data sampling, free from the constraints of fixed grid structures that limit traditional architectures and often require storing complete grid information in memory. This sampling flexibility, when combined with computational optimization techniques like AMP training, provides an effective strategy for enhancing DeepONet’s training efficiency while significantly reducing memory requirements compared to grid-based methods.

4.3. Effects of positional encoding

Building upon the optimal merge network configuration and sampling strategy identified in previous sections (merge Type 1 with 16×16 subsampling with AMP), we investigate the impact of positional encoding (PE) on DeepONet’s prediction accuracy for spatio-temporal flow fields. While PE is a highly effective and well-established technique for coordinate-based networks in computer vision [44,46,52–55], its application has predominantly focused on spatial coordinates. Consequently, clear guidelines for their optimal application to complex spatio-temporal problems within operator learning frameworks like DeepONet are less established. A key motivation for this part of our study, therefore, is to systematically explore its potential effectiveness and specific behaviors when applied to spatio-temporal prediction context. This investigation aims to provide valuable insights and practical guidelines for DeepONet users working at the intersection of these domains (e.g., CFD and computer science). To this end, we systematically

Table 3
Effects of positional encoding strategies and hyperparameters on DeepONet performance.

PE Strategy	Scale Factor (σ)	Mapping Size (m)	Training Hour	Test Error
Without PE	–	–	0.350	17.419
		16	0.355	18.271
		32	0.363	16.751
		64	0.363	17.331
PE(t)	5	16	0.355	17.797
		32	0.365	17.578
		64	0.359	18.065
	10	16	0.358	16.619
		32	0.368	16.100
		64	0.357	17.421
PE(x, y)	1	16	0.398	21.255
		32	0.421	21.512
		64	0.405	20.658
	5	16	0.460	25.196
		32	0.405	21.957
		64	0.422	20.978
	10	16	0.410	117.874
		32	0.454	27.275
		64	0.405	22.090
	1	16	0.356	21.709
		32	0.355	24.981
		64	0.352	24.197
PE(x, y, t)	5	16	0.354	43.161
		32	0.359	32.261
		64	0.351	29.300
	10	16	0.358	81.884
		32	0.356	38.925
		64	0.306	NaN ^a

^a Four out of five models diverged, indicating significant instability in this setting due to high-frequency embedding at spatial coordinates.

explore three distinct PE strategies: (1) **PE(t)**: applying PE only to the temporal coordinate (t); (2) **PE(x, y)**: applying PE only to the spatial coordinates (x, y); and (3) **PE(x, y, t)**: applying PE to all spatio-temporal trunk network inputs (x, y , and t). For each strategy, we evaluate combinations of two key hyperparameters: the scale factor σ controlling the frequency range, and the mapping size m determining the dimension of encoded features. Besides σ and m , all other training settings remain consistent with previous settings. Though sigma values are explored in a greedy manner in this paper, please note that the selection of optimal σ can be aided by the prior knowledge on the dataset: for example, if dominant temporal periods in the flow dynamics are known, σ could be tuned to these frequencies to better capture temporal patterns. However, in more general cases where such prior knowledge is unavailable, a preliminary parametric study, analogous to our investigation summarized in Table 3, is advisable.

The comparison of different PE strategies in Table 3 reveals several important insights about DeepONet's behavior with coordinate encoding. First, it's noteworthy that all models demonstrate broadly similar training times (approximately 0.35–0.46 h with NVIDIA A6000 GPU), indicating that the application of PE does not meaningfully impact computational efficiency. However, the results show a clear distinction between encoding strategies. The temporal-only strategy, PE(t), improves model performance, with the best configuration ($\sigma = 10, m = 32$) achieving a test error of 16.100 compared to 17.419 without PE—a 7.57% improvement. In contrast, applying PE to the spatial coordinates proved to be detrimental in our tests. The strategy of encoding all spatio-temporal inputs, PE(x, y, t), led to deteriorated performance across all configurations. Also, experiments focusing only on spatial encoding, PE(x, y), confirm this finding, as this strategy also consistently yielded higher errors than the baseline without PE (with a minimum test error of 20.658). This deterioration becomes particularly severe with higher scale factors for any strategy involving spatial encoding, culminating in training instability for PE(x, y, t) at $\sigma = 10$ and $m = 64$, where four out of five training runs diverged.

The impact of hyperparameters reveals important considerations for PE implementation. The scale factor σ , which controls the frequency range of the encoding, shows strong influence on model performance. In PE(t), higher σ values generally lead to better performance, suggesting that temporal dynamics in our flow field benefit from higher-frequency encodings. Conversely, for strategies involving spatial coordinates (PE(x, y) and PE(x, y, t)), higher σ values dramatically worsened performance, suggesting that high-frequency spatial embeddings destabilize the training process for this problem. This relationship between performance and σ is highly dependent on the underlying characteristics of the spatio-temporal field, emphasizing the importance of careful hyperparameter tuning for specific applications. Furthermore, the mapping size m demonstrates a non-monotonic relationship with model performance. For instance, in PE(t), increasing m from 32 to 64 consistently degrades performance across all scale factors, indicating that larger

mapping sizes do not necessarily translate to better predictions. This suggests an optimal intermediate dimension for the encoded features that balances expressiveness with model complexity, with $m = 32$ proving to be the optimal value in our case.

In conclusion, our results demonstrate that positional encoding, when properly configured and applied selectively to temporal coordinates, can enhance DeepONet's prediction accuracy without compromising computational efficiency. The 7.57% improvement achieved with PE(t) is a valuable addition for spatio-temporal prediction tasks, requiring no significant architectural or implementation overhead. However, the more critical insight from our systematic study is the finding that only temporal encoding is beneficial, whereas spatial encoding is detrimental for this problem. This stands in contrast to many computer vision applications where PE is successfully applied to spatial coordinates. Therefore, our investigation provides a valuable and non-obvious guideline: for spatio-temporal problems, one must carefully consider which coordinates to encode, as an indiscriminate application of PE can significantly impair model performance.

5. Application of multi-fidelity DeepONet

5.1. Effects of different transfer learning approaches with different low-fidelity dataset

Building upon our optimized DeepONet architecture with temporal positional encoding ($\sigma = 10$, $m = 32$) in Section 4.3, we evaluate the following various MF-DeepONet architectures, all designed with the same model parameter counts to ensure fair comparison:

1. **Lu et al. [29] (Conventional MF-DeepONet):** while the original MF-DeepONet approach proposed by Lu et al. [29] combines both residual learning and input augmentation within a coupled two-DeepONet framework, we strategically implement only the input augmentation aspect. We discard the coupled use of two DeepONet models and residual learning component due to their significant limitation: they fundamentally require identical query points across fidelity levels, which severely restricts the flexible usage of multi-fidelity datasets. Therefore, its implementation uses a sequential training architecture where, following the input augmentation strategy from Lu et al. [29], the output of the pre-trained LF model is used as an additional input for the HF model's trunk network. This architecture still necessitates that both networks execute in sequence during all training and inference operations, requiring much more computational time than our proposed framework.
2. **Proposed:** as proposed in Section 2.2.1, our MF-DeepONet employs fine-tuning where the pre-trained LF DeepONet's branch and trunk networks are frozen, while only the merge network parameters are updated during HF training. This approach preserves valuable LF representations while adapting specifically to HF features. This architecture only requires HF DeepONet during inference stage.
3. **Full-tuning:** all network components (branch, trunk, and merge networks) are simultaneously trainable, potentially compromising the established low-fidelity knowledge while predominantly focusing on high-fidelity feature acquisition. This architecture only requires HF DeepONet during inference stage.
4. **Linear probing:** takes a highly restrictive approach by freezing all layers except the final layer of the merge network. This architecture only requires HF DeepONet during inference stage.

We systematically evaluate these approaches across different LF data configurations, varying both spatial resolution (16×16 , 32×32 , 64×64) and dataset size (50, 100, 200, 300 samples)—note that HF dataset size is fixed as 100. For fair comparison with the single-fidelity DeepONet which uses 800 training epochs, both LF and HF training phases in the multi-fidelity approaches are set to 400 epochs each, ensuring the total number of training epochs remains constant.

The results, presented in Table 4 and also visualized as Fig. 9 for easier comparison of MSE and training hour, reveal several insights about MF-DeepONet performance. First, to properly contextualize the benefits of a multi-fidelity approach, we establish the performance of single-fidelity models. Notably, training exclusively on low-fidelity data—even with a sufficient dataset size—proves insufficient for achieving competitive performance. For instance, single-fidelity models trained on 300 samples of 16×16 , 32×32 , and 64×64 data achieve MSE values of 70.154, 32.669, and 20.223, respectively. While the single-fidelity model trained on 100 high-fidelity samples performs better (MSE 16.100), all these single-fidelity cases are substantially outperformed by our best multi-fidelity model (MSE 9.057). This demonstrates that simply increasing the quantity of low-fidelity data cannot compensate for the fundamental need for high-fidelity information, highlighting the necessity of a multi-fidelity approach.

With these baselines established, we proceed to compare the multi-fidelity methods. Our proposed fine-tuning approach consistently outperforms all other methods, including the input augmentation implementation of the conventional MF-DeepONet (Lu et al. [29]). It shows limited effectiveness, with MSE values ranging from 25.407 to 26.388 for 16×16 resolution data, significantly underperforming even single-fidelity training with high-resolution data (MSE 16.100). Even at higher resolutions (64×64 LF data with 300 samples), this approach achieves an MSE of only 15.940, which is 76% higher than our fine-tuning method's 9.057 MSE. While these inferior results from conventional MF-DeepONet may partly stem from our necessary omission of the residual learning component from the original Lu et al. [29] framework, this design choice was deliberate and essential for our study's objectives. The residual learning approach, while potentially beneficial in certain contexts, fundamentally restricts DeepONet's inherent data flexibility by requiring identical sampling points across fidelity levels—a constraint that severely limits practical applicability in real engineering scenarios where sampling locations often differ between low and high-fidelity simulations (indeed, input function parameters, var_1 and var_2 , are not nested across fidelity levels in this study). Additionally, the sequential architecture of Lu et al. [29]'s approach—requiring both LF and HF DeepONet networks to be executed in sequence during inference—leads to consistently higher computational costs, with training times approximately 25–35% longer than our proposed method across all configurations.

Table 4

Comparison of different MF-DeepONet architectures. Single-fidelity results are shown for reference (e.g., “Single-fidelity (16×16 , 300)” indicates 300 training samples with 16×16 resolution).

Architecture	LF Data Resolution	LF Data Size	Training Hour	MSE
Single-fidelity (16×16 , 300)	—	—	0.476	70.154
Single-fidelity (32×32 , 300)	—	—	1.257	32.669
Single-fidelity (64×64 , 300)	—	—	1.272	20.223
Single-fidelity (128×128 , 100)	—	—	0.368	16.100
Lu et al. [29] (conventional MF-DeepONet)	16×16	50	0.348	26.273
		100	0.411	26.388
		200	0.492	26.123
		300	0.598	25.407
	32×32	50	0.341	27.029
		100	0.399	23.432
		200	0.513	20.694
		300	0.630	18.936
	64×64	50	0.390	25.710
		100	0.407	21.053
		200	0.518	16.628
		300	0.633	15.940
Proposed (fine-tuning)	16×16	50	0.219	19.494
		100	0.268	17.739
		200	0.370	15.238
		300	0.466	15.096
	32×32	50	0.213	19.336
		100	0.269	14.324
		200	0.362	11.791
		300	0.460	10.536
	64×64	50	0.221	17.673
		100	0.272	14.698
		200	0.369	10.782
		300	0.469	9.057
Full-tuning	16×16	50	0.251	21.687
		100	0.301	21.119
		200	0.401	17.893
		300	0.504	17.974
	32×32	50	0.254	22.050
		100	0.303	16.807
		200	0.416	14.772
		300	0.506	13.146
	64×64	50	0.256	20.870
		100	0.316	17.208
		200	0.414	13.612
		300	0.538	11.171
Linear probing	16×16	50	0.176	73.678
		100	0.225	64.520
		200	0.332	60.894
		300	0.433	61.179
	32×32	50	0.175	48.306
		100	0.221	36.800
		200	0.324	32.190
		300	0.429	29.881
	64×64	50	0.181	35.587
		100	0.239	26.301
		200	0.350	21.373
		300	0.466	20.163

Second, fine-tuning consistently outperforms both full-tuning and linear probing across all configurations, demonstrating the effectiveness of our strategy to preserve learned low-fidelity latent representations by branch/trunk networks while adapting only the merge network to high-fidelity features. Particularly noteworthy is that even with the coarsest LF resolution (16×16), fine-tuning achieves better performance compared to single-high-fidelity training (MSE of 15.238 vs 16.100) when provided with sufficient LF

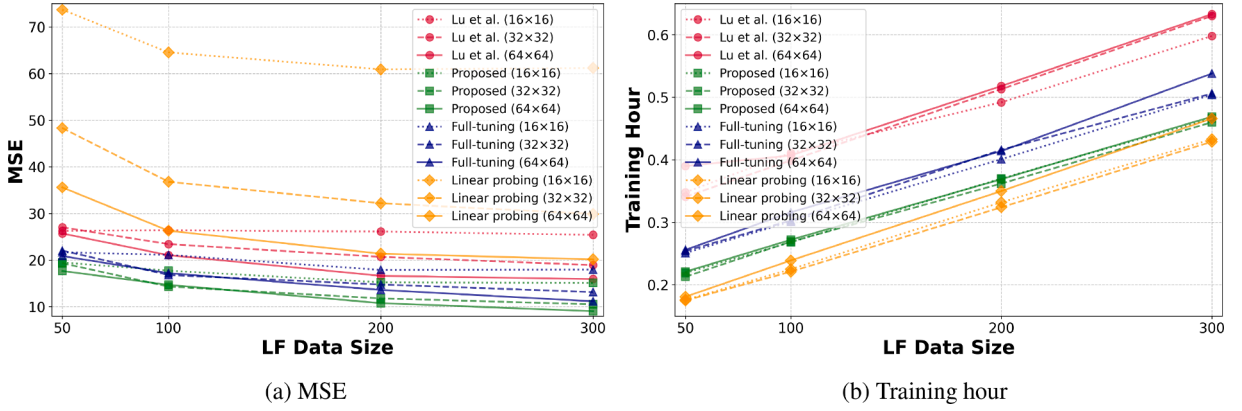


Fig. 9. Performance comparison of conventional DeepONet (Lu et al. [29]), fine-tuning, full-tuning, and linear probing approaches for MF-DeepONet. Results show MSE and training time across different LF data sizes (50–300) and spatial resolutions (16 × 16: dotted, 32 × 32: dashed, 64 × 64: solid lines). This figure illustrates a visual representation of the numerical results presented in Table 4, enabling direct comparison of performance metrics across different model configurations.

data (200 samples). This improvement is unexpected given that the 16 × 16 resolution data exhibits remarkably different flow physics compared to higher resolutions, as evidenced in Fig. 6. While full-tuning provides more flexibility through complete network adaptation, this additional freedom may not be optimal, as it can lead to less efficient use of valuable low-fidelity feature representations learned during pre-training. The superior performance of fine-tuning suggests that the branch and trunk networks effectively capture essential input function parameter and spatio-temporal representations during low-fidelity training, which can be successfully leveraged for high-fidelity predictions through careful adaptation of the merge network alone.

In terms of LF dataset, as expected, increasing the resolution of LF data from 16 × 16 to 64 × 64 leads to consistent improvements in prediction accuracy across all transfer learning approaches. With 300 LF samples at 64 × 64 resolution, fine-tuning achieves an MSE of 9.057, representing a 43.7% improvement over single-high-fidelity training (where MSE is 16.001). Fig. 10 presents their comparative visualization of vorticity field predictions at two different time steps ($t = 0.5s$ and $t = 0.75s$). Our proposed MF-DeepONet with fine-tuning demonstrates notably superior prediction accuracy compared to the single-fidelity approach, particularly in regions highlighted by white dotted boxes. Within these regions, the single-fidelity model exhibits significant deviations from the ground truth, failing to capture the correct vorticity patterns and flow structures. In contrast, our proposed MF-DeepONet successfully reproduces the complex flow features, maintaining fidelity to the true flow dynamics across both temporal snapshots.

Note on how different physics across fidelities affects multi-fidelity performance: The results in Table 4 reveal important insights about knowledge transfer between fidelity levels. Linear probing consistently performs poorly across all configurations, with MSE values substantially higher than both fine-tuning and single-fidelity approaches. This underperformance is scientifically significant—it confirms the substantial differences between low-fidelity and high-fidelity flow physics, as the frozen representations from low-fidelity models inadequately capture high-fidelity features without adaptation. Despite these fundamental differences, our fine-tuning approach demonstrates remarkable effectiveness—even when using coarse 16 × 16 resolution low-fidelity data with markedly different physical characteristics from ground truth physics, the model still outperforms training on HF data alone. Specifically, fine-tuning with 16 × 16 resolution low-fidelity data achieves MSE values of 15.238 (with 200 samples) and 15.096 (with 300 samples), representing improvements of 5.4% and 6.2% respectively over the single-fidelity baseline with 128 × 128 resolution (MSE: 16.100). This confirms that our selective adaptation approach effectively leverages useful information from low-fidelity simulations, even when they exhibit partially different physical characteristics compared to their high-fidelity counterparts. These results also highlight that while MF performance improves when the LF data better approximates HF physics (e.g., LF with 64 × 64 resolution), the proposed framework can still extract meaningful information from LF data of lower fidelity (e.g., LF with 16 × 16 resolution), provided the dataset is sufficiently large. This suggests that implicit correlations between LF and HF domains can still be harnessed through data-driven adaptation. Nevertheless, we still acknowledge that the effectiveness of the MF framework is sensitive to the overall quality of the LF data (Section 2.2.3), and this dependency should be carefully considered when applying the framework to problems with unreliable or weakly correlated LF sources.

Note on computational efficiency of proposed DeepONet over HF simulations: The total training time for all explored models of our proposed MF-DeepONet framework reported in Table 4 is approximately 3.958 h, with each model averaging around 0.33 h of training time. Considering that generation of a single CFD simulation for one HF sample (128 × 128 resolution) takes about 0.1 h, the computational effort to train one MF-DeepONet model is less than generating only four new HF simulations. This comparison highlights that the one-time computational cost of training our surrogate model represents an attractive alternative to generating additional HF simulation data, since the trained DeepONet model can subsequently produce flow field predictions virtually instantaneously, enabling rapid evaluations for downstream tasks such as design optimization, flow control, or uncertainty quantification. This inference efficiency can be especially beneficial in practical scenarios where repeated evaluations are necessary and HF simulations would otherwise become a computational bottleneck.

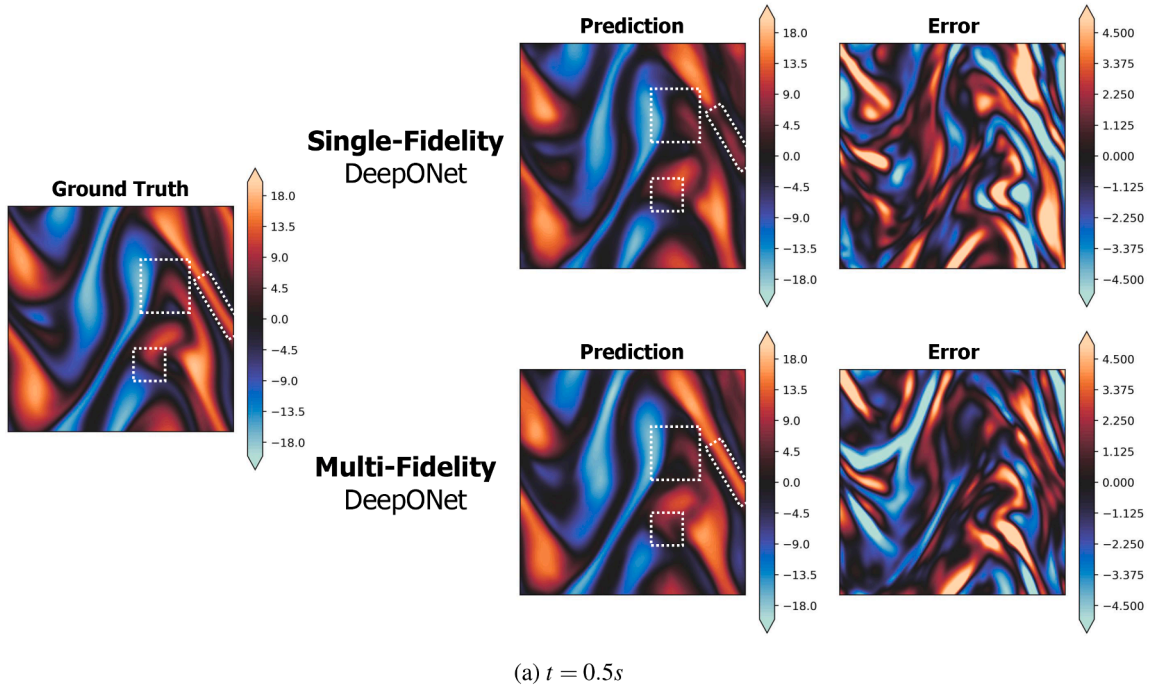
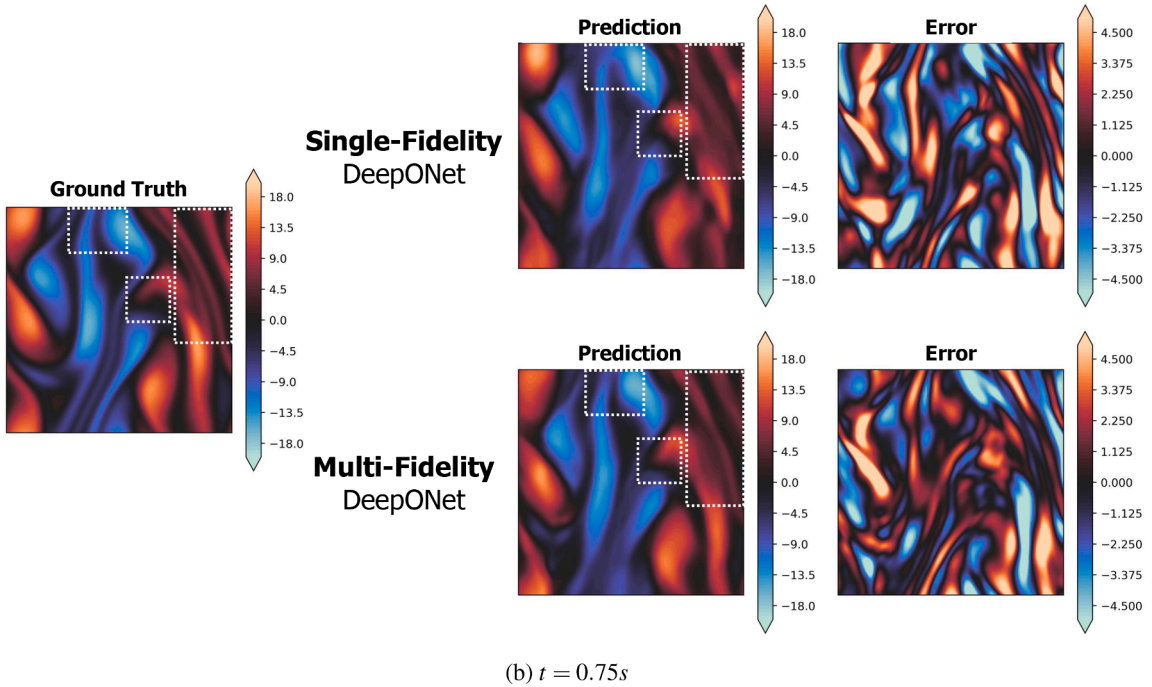
(a) $t = 0.5s$ (b) $t = 0.75s$

Fig. 10. Visual comparison of the predicted vorticity field at $t = 0.5s$ and $t = 0.75s$ between single-fidelity (128x128 resolution) DeepONet and multi-fidelity DeepONet with fine-tuning (with LF data resolution of 64x64 and data size of 300) within test scenario.

5.2. Performance analysis with limited high-fidelity data

To further investigate the potential of our MF-DeepONet framework in scenarios where HF data is scarce, we conduct a systematic study using reduced HF dataset sizes. While our previous analysis in [Section 5.1](#) utilized 100 HF samples, here we examine the model's performance across a range of smaller HF dataset sizes: 20, 40, 60, and 80 samples. This investigation addresses a critical challenge in real-world engineering applications, where HF data is typically extremely limited. Based on our findings from [Section 5.1](#), we focus

Table 5

Performance comparison of different sampling strategies: uniform (Before), proposed time-derivative guided (Time-Deriv. Sampling), and conventional residual-based (Residual).

Metric	Case	Before	Time-Deriv. Sampling	Residual Sampling
MSE	Case 1: LF 32×32 w/ 80 HF	11.759	10.581 (−10.02 %)	10.629 (−9.61 %)
	Case 2: LF 64×64 w/ 60 HF	11.364	9.008 (−20.73 %)	9.121 (−19.74 %)
Training hour	Case 1: LF 32×32 w/ 80 HF	0.469	0.505 (+7.68 %)	0.555 (+18.34 %)
	Case 2: LF 64×64 w/ 60 HF	0.475	0.526 (+10.74 %)	0.562 (+18.32 %)

on the more effective LF resolutions of 32×32 and 64×64 , excluding the 16×16 resolution due to its relatively limited performance benefits. For each LF resolution, we explore two different LF dataset sizes (200 and 300 samples) to understand how the quantity of LF data affects the model's ability to compensate for limited HF data. This systematic approach will allow one to establish practical guidelines for efficiently allocating computational resources between LF and HF simulations in practical applications.

The results shown in Fig. 11 reveal several important insights about the MF-DeepONet's performance with limited HF data. First, examining the MSE trends (Fig. 11a), we observe a consistent pattern of improved prediction accuracy as the number of HF samples increases across all configurations. The 64×64 resolution configurations consistently outperform their 32×32 counterparts, with the 64×64 with 300 samples combination achieving the best performance across all HF dataset sizes. The training time analysis (Fig. 11b) reveals an expectable trade-off. Configurations with 300 LF samples consistently require longer training times compared to their 200 LF sample counterparts due to the longer training time required in LF deepONet. However, this increased computational cost appears justified by the improved accuracy, particularly for the 64×64 resolution cases.

6. Physics-guided subsampling for enhancing multi-fidelity DeepONet

6.1. Effects of physics-guided subsampling

Based on our analysis in Section 5.2, we will further investigate two configurations of our fine-tuned MF-DeepONet framework (highlighted in Fig. 11a as yellow circles with black dashed edges) that achieve an moderate balance between prediction accuracy and high-fidelity data efficiency:

1. **Case 1:** MF-DeepONet with 32×32 LF resolution (300 samples) and 80 HF samples
2. **Case 2:** MF-DeepONet with 64×64 LF resolution (300 samples) and 60 HF samples

For these two configurations, we analyze the effectiveness of our physics-guided (specifically, time-derivative guided) sampling strategy in practice. This physics-guided approach, formalized in Algorithm 1 in Section 2.2.2, uses a pre-trained LF DeepONet to identify regions with pronounced temporal dynamics. The purpose of this strategy is to make the training of the HF-DeepONet efficient by guiding the selection of a sparse but physically informative subset of spatial points from the full, already available HF simulation dataset, thereby enhancing data utilization efficiency during model training.

In our experiments, we first analyze the effectiveness of our proposed time-derivative guided subsampling. For a more comprehensive comparison, we also present the results from a conventional residual-based sampling strategy, which is a well-established technique in the domain of physics-informed neural networks [56]. This method operates by identifying and sampling points where the errors (residuals) between the LF model's predictions and the HF ground truth data are large. To ensure a fair comparison between

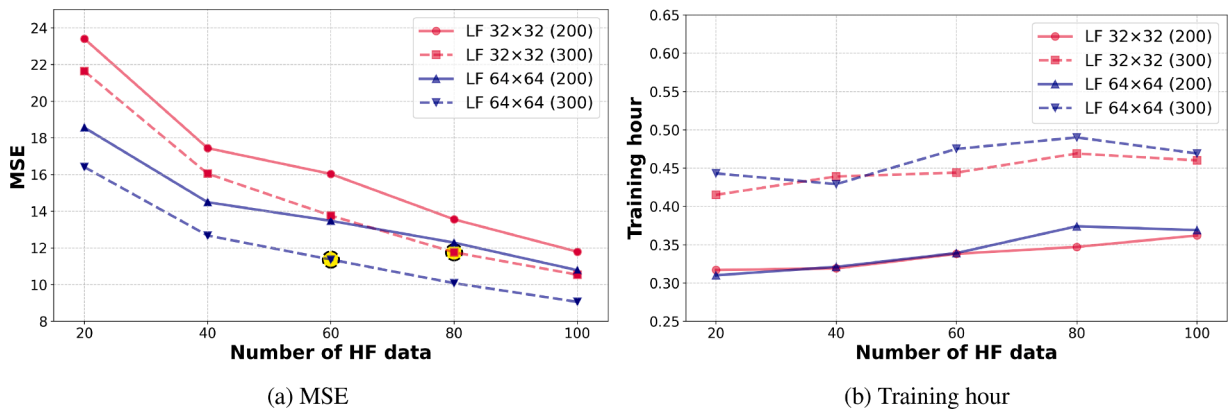


Fig. 11. MF-DeepONet performance with limited high-fidelity data: different resolutions of LF data with different LF data sizes are explored. The label “LF 32×32 (200)” represents a case where LF data is from a 32×32 resolution with a dataset size of 200—all other labels follow the same notation.

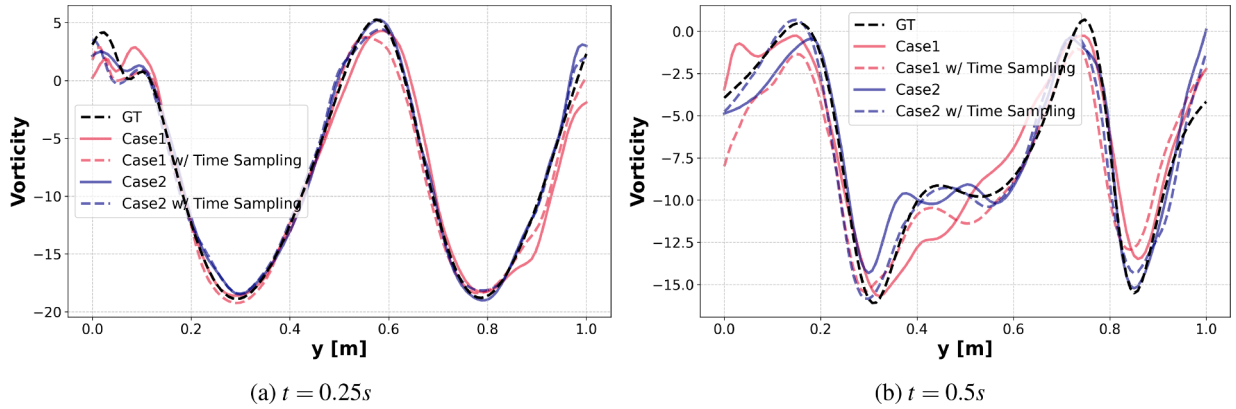


Fig. 12. Comparison of vorticity profiles from ground truth (GT) and MF-DeepONet predictions with and without time-derivative guided subsampling. At early stages ($t = 0.25s$), all models show comparable performance, but at later times ($t = 0.5s$), models with time-derivative subsampling demonstrate superior accuracy in capturing complex flow dynamics. Case 1: 32×32 LF (300 samples) with 80 HF samples; Case 2: 64×64 LF (300 samples) with 60 HF samples.

these two guided approaches, we set the sampling ratio to $r = 0.1$ for both, meaning that 10 % of the HF spatial points are strategically selected based on the respective criterion, while the remaining 90 % are sampled uniformly at random to maintain spatial coverage.

The results, detailed in Table 5, demonstrate that while both methods significantly improve accuracy over the baseline, our proposed time-derivative subsampling is slightly superior in both accuracy and computational cost. Our physics-guided approach achieves MSE reductions of 10.02 % for Case 1 and 20.73 % for Case 2. In comparison, the residual-based method yields slightly smaller MSE reductions of 9.61 % and 19.74 % for the same cases. Furthermore, our approach is more computationally efficient, with an 8–11 % increase in training time overhead, compared to the more significant 18 % increase required for residual sampling. This additional overhead for both methods is primarily attributed to the calculations required to guide the sampling (i.e., automatic differentiation for temporal derivatives or residual computation). The greater improvement seen in the Case 2 configurations for both methods indicates that a higher-resolution LF data produces a more accurate LF DeepONet model, which in turn provides more reliable guidance.

The above quantitative improvements are further supported by the qualitative analysis shown in Fig. 12, which compares vorticity profiles at different time steps. At $t = 0.25s$ (Fig. 12a), all models show similar prediction capabilities, closely following the ground truth profile. This comparable performance in the early stage is expected, as the flow field remains relatively simple and closer to the initial conditions. However, at $t = 0.5s$ (Fig. 12b), clear differences emerge in the models' ability to capture complex flow dynamics. This later time step exhibits notably more complicated flow development with pronounced temporal variations. Indeed, particularly in the middle region ($y = 0.3m$ to $0.7m$), models trained with time-derivative subsampling (presented as dashed lines) more accurately reproduce the local fluctuations and subtle variations in the vorticity profile. This superior performance demonstrates that our approach automatically identifies and focuses computational resources on regions with significant temporal dynamics, enhancing prediction accuracy exactly where HF DeepONet should focus. In contrast, models without sampling tend to oversimplify these local features, capturing only the overall trend while missing important flow structure details. These results confirm that our physics-guided subsampling strategy effectively targets time-sensitive regions of the flow field ($t = 0.5s$ than $t = 0.25s$), showing particular advantage in predicting complex time-dependent dynamics.

6.2. Reduced high-fidelity data requirement through physics-guided subsampling

The time-derivative guided subsampling strategy demonstrates remarkable effectiveness, achieving superior predictive accuracy compared to conventional uniform sampling when using the same amount of HF data (Section 6.1). Therefore, in this section, we investigate how this approach can reduce the required HF dataset size while maintaining comparable accuracy to models trained without time-derivative subsampling—quantifying the potential data efficiency gains enabled by our physics-guided subsampling method. For MF-DeepONet using 64×64 LF resolution data (Case 2), our sampling approach achieves an MSE of 9.008 with only 60 HF samples, surpassing the performance of models trained on the full 100 HF samples with uniform sampling (MSE 9.057, marked as bold in Table 4). This achievement is particularly significant as it represents both improved accuracy and a 40 % reduction in required HF data. Similarly, with 32×32 LF resolution data (Case 1), our time-based sampling achieves prediction accuracy (MSE 10.581) using only 80 HF samples, effectively matching the performance of training with the complete HF dataset and uniform sampling (MSE 10.536, marked as bold in Table 4).

Fig. 13 provides a visual comparison between the uniform-sampling-based multi-fidelity model trained with the 100 HF dataset and the multi-fidelity model coupled with time-derivative guided subsampling approach using only 60 HF dataset. At both time steps ($t = 0.5s$ and $t = 0.75s$), we observe that both models produce visually comparable flow field predictions that closely match the ground truth. The error distributions demonstrate that, despite using significantly less HF data (only 60 %), the time-derivative

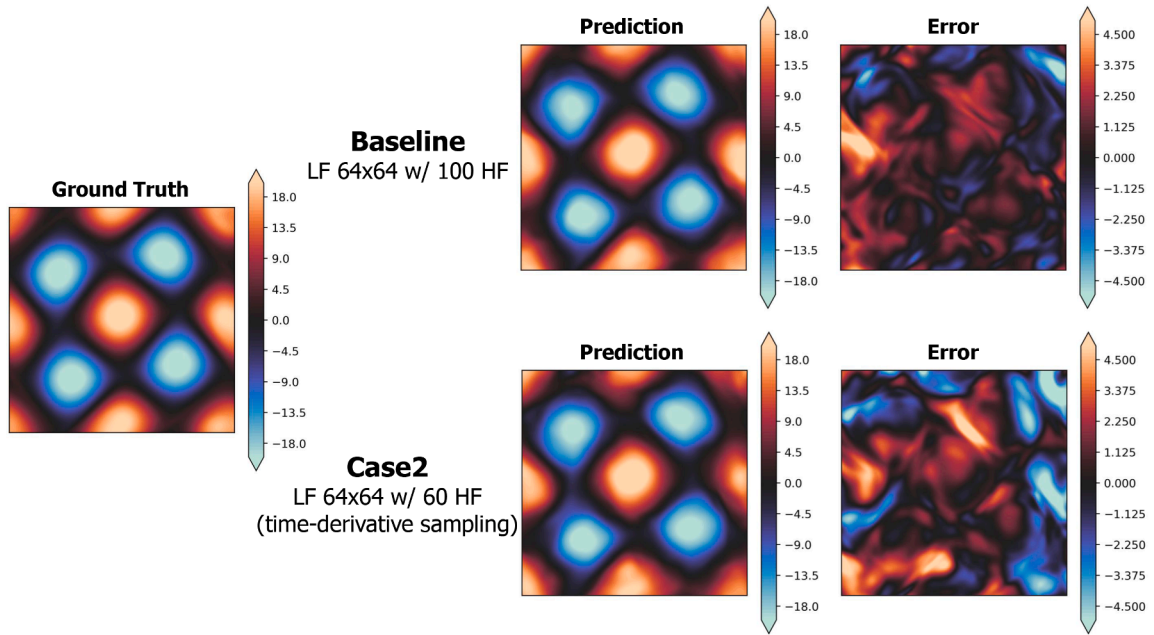
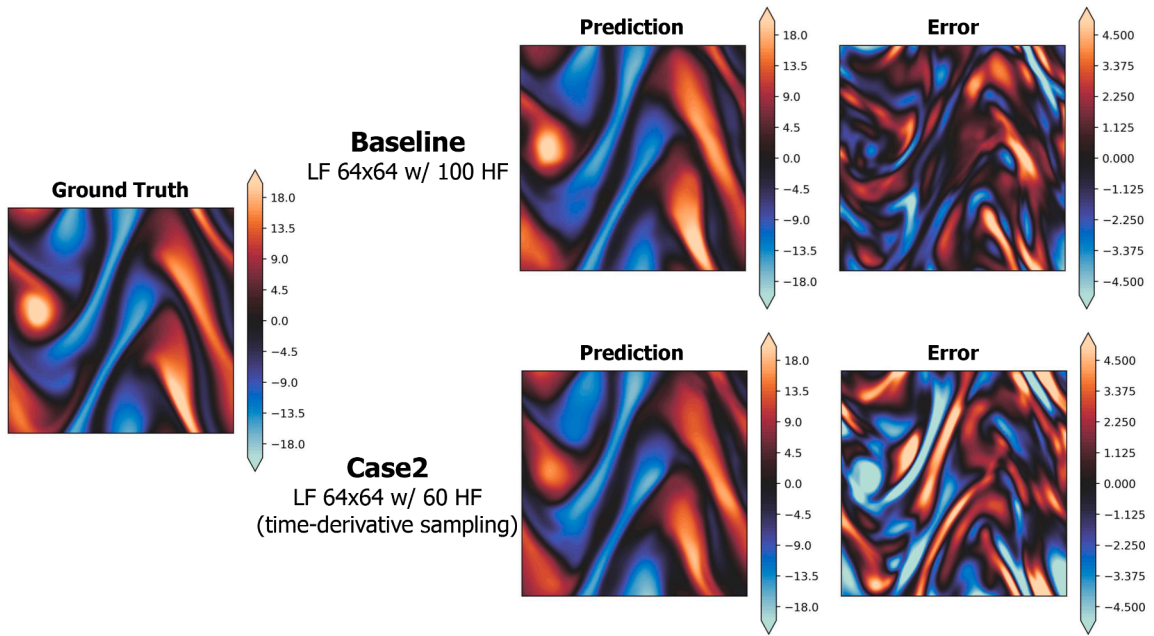


Fig. 13. Visual comparison of the predicted vorticity field at $t = 0.5s$ and $t = 0.75s$ between two models: 1) baseline uniform-sampling-based MF-DeepONet with LF 64×64 and 100 HF samples, and 2) MF-DeepONet with LF 64×64 and only 60 HF samples guided by time-derivative subsampling. Despite using 40% fewer HF samples, the model with time-derivative subsampling maintains comparable prediction quality, with only minor differences in error patterns.

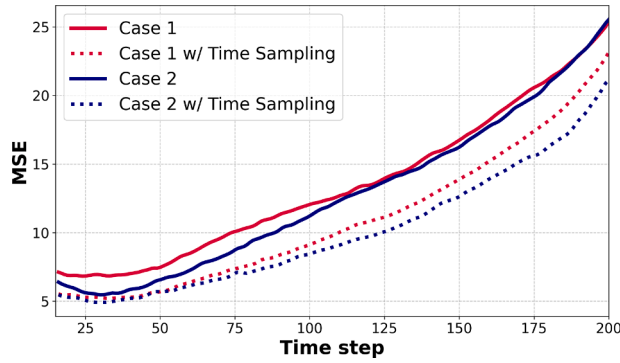


Fig. 14. Comparison of MSE behavior over time steps (200 steps stands for a second) for Case 1 and Case 2, with and without time-derivative guided subsampling. The results clearly demonstrate that the time-derivative guided subsampling effectively reduces error over time, thereby significantly improving prediction stability and accuracy.

subsampling approach maintains similar prediction quality to the uniform-sampling model. While there are subtle differences in the error patterns—particularly visible in certain regions at $t = 0.75s$ where the proposed physics-guided subsampling approach shows slightly higher localized errors—these minor differences in error patterns are insignificant when considering the substantial 40 % reduction in HF data requirements achieved by our sampling approach. This visual evidence reinforces our quantitative findings that strategic selection of high-fidelity training points based on temporal dynamics enables efficient utilization of limited HF data with minimal degradation in prediction quality.

Fig. 14 presents an analysis of error behavior of the test data over time for Cases 1 and 2, comparing scenarios with and without time-derivative guided subsampling. It is clear from the plot that incorporating time-derivative guided subsampling mitigates error increase as temporal prediction progresses for both cases. This highlights that the physics-guided, time-derivative subsampling not only enhances accuracy at individual time steps but also substantially limits error propagation over extended prediction intervals. Such improved long-term predictive stability is particularly valuable for industrial applications where accuracy over prolonged simulation times is crucial.

Note on the extensibility of the proposed physics-guided subsampling approach: While our sampling strategy leverages temporal derivatives as the key physical indicator, the MF-DeepONet framework itself is not confined to this specific sampling approach. Temporal derivatives serve as an effective physics-informed criterion for identifying critical regions in the examples studied here, capturing fundamental characteristics where rapid changes often indicate important physical transitions. However, the modular nature of our framework allows for integration with various alternative sampling techniques tailored to specific applications. For instance, future work could explore uncertainty-driven sampling for stochastic systems. This flexibility represents a clear advantage of our approach—the underlying MF-DeepONet architecture can be coupled with domain-specific sampling strategies while maintaining its core multi-fidelity benefits. This opens numerous avenues for researchers to develop specialized sampling techniques optimized for their particular physical systems, potentially yielding even greater efficiency gains beyond what we've demonstrated with temporal derivatives in this study.

7. Further validation with more challenging datasets

To further consolidate the robustness and superior performance of our proposed MF-DeepONet framework, particularly in scenarios demanding higher precision and involving more local flow dynamics, we introduce a new set of more challenging datasets. These datasets are designed to incorporate higher spatial resolutions and more complex initial conditions compared to those presented in Section 3. This section first validates our MF-DeepONet framework against conventional approaches using new datasets. Subsequently, we conduct a detailed investigation of our proposed time-derivative method, comparing it with a conventional residual-based approach to assess its efficacy under these more demanding conditions.

7.1. Generation of higher-resolution and more complex flow field datasets

The generation process for these new, more challenging datasets largely follows the methodology outlined in Section 3, utilizing the same pseudo-spectral solver for the incompressible Navier-Stokes equations in a two-dimensional periodic domain $[0, 1m] \times [0, 1m]$ with a Reynolds number of $Re = 1,000$. However, two key aspects are modified:

1. **More complex initial conditions:** The parameters var_1 and var_2 defining the initial velocity field components (Eq. (6)) are now sampled from a uniform distribution in the range $\mathcal{U}[5, 7]$ (previously $\mathcal{U}[1, 3]$). This change introduces higher spatial frequencies into the initial state of the flow, leading to more complex and rapidly evolving flow structures.
2. **Higher spatial resolutions:** The fidelity levels are increased accordingly as flow characteristics become complicated. For the HF dataset, we now generate 100 training samples and 50 test samples on a finer uniform grid of 256×256 resolution (previously

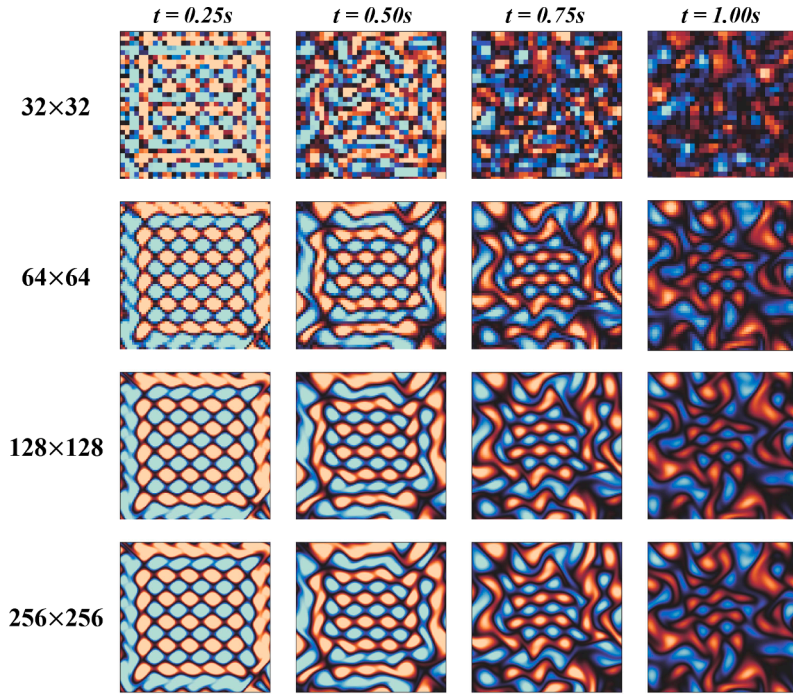


Fig. 15. Effects of grid resolution on vorticity field evolution with fixed parameters $var_1 = 6.4$ and $var_2 = 6.5$ (contours represent vorticity values).

128 $\times$ 128). For the LF datasets, we generate 300 training samples for each of three upgraded resolutions: 32 $\times$ 32, 64 $\times$ 64, and 128 $\times$ 128 (previously 16 $\times$ 16, 32 $\times$ 32, and 64 $\times$ 64).

These modifications therefore increase the computational cost of data generation. While HF 256 $\times$ 256 data requires approximately 1800 s per sample, the LF data requires 75, 120, and 330 s for 32 $\times$ 32, 64 $\times$ 64, and 128 $\times$ 128 resolutions, respectively. In total, generating the HF dataset (100 training samples) requires 50 h, whereas generating each LF dataset (300 samples) requires 6.25, 10, and 27.5 h, respectively.

The vorticity fields generated using these more challenging conditions are visualized across different resolutions in Fig. 15: this figure clearly illustrates the increased complexity compared to the initial dataset presented in Fig. 6. The higher values used for var_1 and var_2 in the initial conditions (Eq. (6)) result in a greater number of initial vorticity structures and consequently, a more intricate mixing process as the flow evolves. Similar to observations with the previous dataset, Fig. 15 shows that the lowest LF resolution (32 $\times$ 32) struggles to represent the detailed flow structures visible in the HF (256 $\times$ 256) simulation, particularly at later time steps. In contrast, the intermediate LF resolutions (64 $\times$ 64 and 128 $\times$ 128) capture the essential macroscopic flow characteristics faithfully.

7.2. Proposed MF-DeepONet framework performance

Here, we validate the performance of our proposed MF-DeepONet framework against the conventional approach by Lu et al. [29]. Since our initial results in Section 5.1 conclusively showed that our fine-tuning strategy was superior to both full-tuning and linear probing, we focus this validation on our best-performing method (fine-tuning) to ensure a direct comparison against the primary baseline (Lu et al. [29]). The experimental setup, including network architectures, largely mirrors that described in Section 5.1. However, to better leverage the more complex flow physics, the LF DeepONet is now trained for 800 epochs, while the subsequent HF fine-tuning phase remains at 400 epochs.

The results, presented in Table 6, again confirm the robustness of our proposed framework under these more demanding conditions. It is evident that the conventional approach by Lu et al. [29] struggles to achieve a reasonable prediction error across all its configurations, indicating potential limitations when applied to such complex flow physics. In contrast, our proposed MF-DeepONet framework demonstrates much better performance, particularly when leveraging higher-quality LF data. Specifically, the four configurations using LF data at 64 $\times$ 64 and 128 $\times$ 128 resolutions with larger dataset sizes (200 and 300 samples) yield the most accurate and reasonable results. This highlights that the success of our framework is pronounced when the pre-training phase is supported by LF datasets that are both reasonably accurate in resolution and sufficient in quantity, enabling the model to effectively learn and transfer essential flow characteristics for high-fidelity prediction.

7.3. Comparative analysis of physics-guided subsampling strategy

We further investigate the efficacy of our proposed time-derivative guided subsampling strategy using these new challenging datasets, benchmarking it against the conventional residual-based sampling approach [56]. We apply these two subsampling methods to the four most accurate configurations identified in Table 6 (highlighted in bold). As with our previous sampling experiments, we use a strategic sampling ratio of $r = 0.1$.

The performance comparison of these sampling strategies is presented in Table 7. While both subsampling methods offer comparable performance when using the 64x64 LF data, our physics-guided time-derivative strategy demonstrates a clear advantage when a higher-quality LF model is employed. Notably, for the case using 128x128 LF data with 300 samples, time-derivative subsampling yields a 14.20 % error reduction, which is more effective than the 11.63 % reduction from the residual-based method, while both have similar computational overhead. When the guiding LF model is of moderate quality (e.g., 64x64), its predictions contain more pronounced inaccuracies, making the error-driven residual sampling an effective corrective strategy. However, as the LF model becomes more accurate (e.g., 128x128), its ability to reliably identify physically dynamic regions becomes a more powerful form of guidance. In this regime, focusing on these areas of high temporal activity proves to be a more effective strategy for capturing complex dynamics than simply targeting the largest remaining (and now smaller) errors.

Table 6

Comparison of different MF-DeepONet architectures for new dataset. Single-fidelity results are shown for reference (e.g., “Single-fidelity (32×32, 300)” indicates 32×32 resolution with 300 samples).

Architecture	LF Data Resolution	LF Data Size	Training Hour	MSE
Single-fidelity (32×32, 300)	—	—	1.313	165.292
Single-fidelity (64×64, 300)	—	—	1.347	414.936
Single-fidelity (128×128, 300)	—	—	1.408	45.075
Single-fidelity (256×256, 100)	—	—	0.430	15.918
Lu et al. [29] (conventional MF-DeepONet)	32×32	50	0.498	50.228
		100	0.550	60.736
		200	0.770	50.059
		300	0.976	55.436
	64×64	50	0.447	51.301
		100	0.564	52.415
		200	0.831	50.653
		300	1.019	49.644
	128×128	50	0.469	50.367
		100	0.605	48.326
		200	0.809	39.131
		300	1.054	32.995
	32×32	50	0.598	29.997
		100	0.686	25.242
		200	0.915	23.959
		300	1.117	20.610
	64×64	50	0.576	30.048
		100	0.715	23.012
		200	0.885	13.919
		300	1.058	12.028
	128×128	50	0.586	23.745
		100	0.708	20.957
		200	0.914	18.302
		300	1.111	12.364

Table 7

Performance comparison of different sampling strategies on the new datasets. “Before” refers to training with uniform subsampling.

Metric	Case	Before	Time-Deriv. Sampling	Residual Sampling
MSE	LF 64x64 w/ 200 samples	13.919	14.484 (+4.06 %)	14.482 (+4.04 %)
	LF 64x64 w/ 300 samples	12.028	11.929 (−0.82 %)	11.889 (−1.16 %)
	LF 128x128 w/ 200 samples	18.302	18.147 (−0.85 %)	18.386 (+0.46 %)
	LF 128x128 w/ 300 samples	12.364	10.609 (−14.20 %)	10.926 (−11.63 %)
Training hour	LF 64x64 w/ 200 samples	0.885	1.063 (+20.11 %)	1.069 (+20.79 %)
	LF 64x64 w/ 300 samples	1.058	1.279 (+20.89 %)	1.276 (+20.60 %)
	LF 128x128 w/ 200 samples	0.914	1.097 (+19.91 %)	1.107 (+21.11 %)
	LF 128x128 w/ 300 samples	1.111	1.331 (+19.80 %)	1.337 (+20.34 %)

8. Conclusion

This study presents a novel multi-fidelity DeepONet (MF-DeepONet) framework for efficient spatio-temporal flow field prediction with reduced high-fidelity data requirements. Through strategic architectural enhancements and physics-guided subsampling, we address critical challenges in operator learning for fluid dynamics applications. Our merge network architecture enables more complex feature interactions between operator and spatio-temporal coordinate spaces, achieving a 50.4 % reduction in prediction error compared to traditional dot-product approaches. The implementation of temporal positional encoding and efficient point-based sampling strategies further improves prediction accuracy by 7.57 % while dramatically reducing training time by 96 %. The transfer learning-based multi-fidelity framework leverages knowledge from pre-trained low-fidelity models to guide high-fidelity predictions, demonstrating a 43.7 % improvement in accuracy compared to single-fidelity training. Most significantly, our time-derivative guided subsampling strategy demonstrates two key advantages: (1) it reduces prediction error by 20.7 % compared to conventional uniform sampling when using the same amount of high-fidelity data, and (2) it achieves prediction accuracy comparable to a model trained on the complete set of available high-fidelity data, while using only 60 % of those training data—a crucial advancement for computationally expensive simulations. The robustness of these contributions is further validated on a second, more complex dataset with higher spatial resolutions and more intricate flow physics, where our proposed MF-DeepONet framework and time-derivative subsampling strategy continued to demonstrate superior or competitive performance against conventional benchmarks.

Despite these promising results, our framework also has limitations that warrant further investigation. First, the current transfer learning approach—which completely freezes branch and trunk networks—may be overly restrictive; more flexible layer-freezing strategies based on data similarity metrics could potentially yield further improvements. Additionally, while the time-derivative guided subsampling strategy proved effective for our flow problems, it requires broader validation across diverse physical systems with different temporal characteristics. Furthermore, its reliance on automatic differentiation introduces computational overhead, presenting opportunities for developing more efficient sampling implementations. Lastly, for broader industrial applicability, extending the framework to accommodate irregular domains, complex geometries, and moving boundaries represents an important direction for future work.

Future research will focus on several key areas to build upon the findings of this study. First, to improve the multi-fidelity framework itself, an automated layer-wise similarity analysis between low-fidelity and high-fidelity features could enable more nuanced transfer learning approaches with selective network component freezing. The observed training instability in some cases—attributable to the initial training of LF-DeepONet without guidance from HF data—highlights the need for refining the training strategy, which could serve as a promising direction for future research. Another important extension would be to investigate the robustness of MF-DeepONet when the LF and HF datasets are less strongly correlated, such as cases involving different Reynolds numbers or slight geometric domain changes. This would clarify the level of physical similarity required for effective transfer and broaden the framework's applicability in real-world scenarios. Furthermore, to enhance predictive robustness for time-evolving problems, future work will investigate the incorporation of physics-informed loss functions that more explicitly enforce the underlying Navier-Stokes dynamics during training. Second, to advance the sampling strategy, research can focus on developing physics-guided subsampling criteria beyond temporal derivatives—such as incorporating geometric information—to improve cross-domain generalizability and enable effective point selection in complex domains. The computational efficiency of these methods could also be enhanced through surrogate-based derivative estimation or alternative physics-guided metrics that avoid expensive automatic differentiation calculations. These advancements would further establish MF-DeepONet as a practical solution for real-time flow prediction in data-constrained environments.

CRedit authorship contribution statement

Sunwoong Yang: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization; **Youngkyu Lee:** Supervision, Methodology; **Namwoo Kang:** Supervision, Project administration, Funding acquisition.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the [Ministry of Science and ICT of Korea](#) grant (No. 2022-0-00969, No. 2022-0-00986, and No. RS-2024-00355857), the [Ministry of Trade, Industry & Energy](#) grant (RS-2024-00410810, RS-2025-02317327), and the [National Research Council of Science & Technology](#) (NST) grant by the Korea government (MSIT) (No. GTL24031-000).

Appendix A. Automatic mixed precision (AMP)

AMP is a method that blends half-precision and full-precision computations to accelerate training and reduce memory usage, while maintaining stability. In simpler terms, it uses smaller data types (for example, 16-bit floats) during most calculations for speed and efficiency, then switches to larger data types (like 32-bit floats) for operations that risk losing too much detail. This strategy lowers the amount of memory required and speeds up math operations on modern hardware.

A crucial part of AMP is a safeguard mechanism that dynamically scales gradients to avoid floating-point overflow or underflow. In practice, a scaling factor is applied to gradients during backpropagation. After the updates are computed, this factor is reversed before adjusting the model's parameters. If the training process detects an overflow—meaning the gradients became too large—the scaling factor is automatically reduced for the next round of updates, preserving numerical stability. Additional insights into mixed precision and gradient scaling can be found through sources such as the NVIDIA Developer Blog [57].

References

- [1] Q. Wang, J.S. Hesthaven, D. Ray, Non-intrusive reduced order modeling of unsteady flows using artificial neural networks with application to a combustion problem, *J. Comput. Phys.* 384 (2019) 289–307.
- [2] S. Yang, S. Lee, K. Yee, Inverse design optimization framework via a two-step deep learning approach: application to a wind turbine airfoil, *Eng. Comput.* 39 (3) (2023) 2239–2255.
- [3] Y.-E. Kang, S. Yang, K. Yee, Physics-aware reduced-order modeling of transonic flow via β -variational autoencoder, *Phys. Fluids* 34 (7) (2022).
- [4] R. Maulik, B. Lusch, P. Balaprakash, Reduced-order modeling of advection-dominated systems with recurrent neural networks and convolutional autoencoders, *Phys. Fluids* 33 (3) (2021).
- [5] K. Hasegawa, K. Fukami, T. Murata, K. Fukagata, Machine-learning-based reduced-order modeling for unsteady flows around bluff bodies of various shapes, *Theor. Comput. Fluid Dyn.* 34 (2020) 367–383.
- [6] L. Gastoni, A. Güemes, A. Ianiro, S. Discetti, P. Schlatter, H. Azizpour, R. Vinuesa, Convolutional-network models to predict wall-bounded turbulence from wall quantities, *J. Fluid Mech.* 928 (2021) A27.
- [7] J.-W. Hu, W.-W. Zhang, Mesh-Conv: convolution operator with mesh resolution independence for flow field modeling, *J. Comput. Phys.* 452 (2022) 110896.
- [8] H. Eivazi, S. Le Clainche, S. Hoyas, R. Vinuesa, Towards extraction of orthogonal and parsimonious non-linear modes from turbulent flows, *Expert Syst. Appl.* 202 (2022) 117038.
- [9] T. Pfaff, M. Fortunato, A. Sanchez-Gonzalez, P.W. Battaglia, Learning mesh-based simulation with graph networks, (2020). arXiv preprint arXiv:2010.03409
- [10] S. Yang, R. Vinuesa, N. Kang, Towards robust spatio-temporal auto-regressive prediction: adams-bashforth time integration with adaptive multi-step rollout, (2024). arXiv preprint arXiv:2412.05657
- [11] J. Kim, J. Park, N. Kim, Y. Yu, K. Chang, C.-S. Woo, S. Yang, N. Kang, Physics-constrained graph neural networks for spatio-temporal prediction of drop impact on OLED display panels, (2024). arXiv preprint arXiv:2411.01848
- [12] S. Yang, R. Vinuesa, N. Kang, Enhancing graph U-Nets for mesh-agnostic spatio-temporal flow prediction, (2024). arXiv preprint arXiv:2406.03789
- [13] M.A. Nabian, X-MeshGraphNet: scalable multi-scale graph neural networks for physics simulation, (2024). 2411.17164
- [14] L. Lu, P. Jin, G. Pang, Z. Zhang, G.E. Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, *Nat. Mach. Intell.* 3 (3) (2021) 218–229.
- [15] J. He, S. Kushwaha, J. Park, S. Koric, D. Abueidda, I. Jasiuk, Predictions of transient vector solution fields with sequential deep operator network, *Acta Mech.* 235 (8) (2024) 5257–5272.
- [16] H. Bai, Z. Wang, X. Chu, J. Deng, X. Bian, Data-driven modeling of unsteady flow based on deep operator network, *Phys. Fluids* 36 (6) (2024).
- [17] L. Lu, X. Meng, S. Cai, Z. Mao, S. Goswami, Z. Zhang, G.E. Karniadakis, A comprehensive and fair comparison of two neural operators (with practical extensions) based on fair data, *Comput. Methods Appl. Mech. Eng.* 393 (2022) 114778.
- [18] A. Taassob, A. Kumar, K.M. Gitushi, R. Ranade, T. Echecki, A PINN-DeepONet framework for extracting turbulent combustion closure from multiscale measurements, *Comput. Methods Appl. Mech. Eng.* 429 (2024) 117163.
- [19] K. Shukla, J.D. Toscano, Z. Wang, Z. Zou, G.E. Karniadakis, A comprehensive and fair comparison between MLP and KAN representations for differential equations and operator networks, *Comput. Methods Appl. Mech. Eng.* 431 (2024) 117290.
- [20] K. Kontolati, S. Goswami, G. Em Karniadakis, M.D. Shields, Learning nonlinear operators in latent spaces for real-time predictions of complex dynamics in physical systems, *Nat. Commun.* 15 (1) (2024) 5101.
- [21] S. Karumuri, L. Graham-Brady, S. Goswami, Physics-informed latent neural operator for real-time predictions of complex physical systems, (2025). arXiv preprint arXiv:2501.08428
- [22] H. Li, Y. Miao, Z.S. Khodaei, M.H. Aliabadi, An architectural analysis of DeepONet and a general extension of the physics-informed DeepONet model on solving nonlinear parametric partial differential equations, *Neurocomputing* 611 (2025) 128675.
- [23] Z.-H. Han, S. Götz, Hierarchical kriging model for variable-fidelity surrogate modeling, *AIAA J.* 50 (9) (2012) 1885–1896.
- [24] X. Meng, Z. Wang, D. Fan, M.S. Triantafyllou, G.E. Karniadakis, A fast multi-fidelity method with uncertainty quantification for complex data correlations: application to vortex-induced vibrations of marine risers, *Comput. Methods Appl. Mech. Eng.* 386 (2021) 114212.
- [25] S. Yang, H. Kim, Y. Hong, K. Yee, R. Maulik, N. Kang, Data-driven physics-informed neural networks: a digital twin perspective, *Comput. Methods Appl. Mech. Eng.* 428 (2024) 117075.
- [26] X. Zhang, F. Xie, T. Ji, Z. Zhu, Y. Zheng, Multi-fidelity deep neural network surrogate model for aerodynamic shape optimization, *Comput. Methods Appl. Mech. Eng.* 373 (2021) 113485.
- [27] S. Yang, K. Yee, Design rule extraction using multi-fidelity surrogate model for unmanned combat aerial vehicles, *J. Aircr.* 59 (4) (2022) 977–991.
- [28] S. Yang, K. Yee, Comment on “novel approach for selecting low-fidelity scale factor in multifidelity metamodeling”, *AIAA J.* 60 (4) (2022) 2713–2715.
- [29] L. Lu, R. Pestourie, S.G. Johnson, G. Romano, Multifidelity deep neural operators for efficient learning of partial differential equations with application to fast inverse design of nanoscale heat transport, *Phys. Rev. Res.* 4 (2) (2022) 023210.
- [30] S. De, M. Reynolds, M. Hassanaly, R.N. King, A. Doostan, Bi-fidelity modeling of uncertain and partially unknown systems using DeepONets, *Comput. Mech.* 71 (6) (2023) 1251–1267.
- [31] N. Demo, M. Tezzele, G. Rozza, A DeepONet multi-fidelity approach for residual learning in reduced order modeling, *Adv. Model. Simul. Eng. Sci.* 10 (1) (2023) 12.
- [32] A.A. Howard, M. Perego, G.E. Karniadakis, P. Stinis, Multifidelity deep operator networks for data-driven and physics-informed problems, *J. Comput. Phys.* 493 (2023) 112462.
- [33] X. Meng, G.E. Karniadakis, A composite neural network that learns from multi-fidelity data: application to function approximation and inverse PDE problems, *J. Comput. Phys.* 401 (2020) 109020.
- [34] C. Xu, B.T. Cao, Y. Yuan, G. Meschke, A multi-fidelity deep operator network (DeepONet) for fusing simulation and monitoring data: application to real-time settlement prediction during tunnel construction, *Eng. Appl. Artif. Intell.* 133 (2024) 108156.
- [35] S. Panigrahi, A. Nanda, T. Swarnkar, A survey on transfer learning, in: *Intelligent and Cloud Computing: Proceedings of ICICC 2019*, Volume 1, Springer, 2020, pp. 781–789.
- [36] D.H. Song, D.M. Tartakovsky, Transfer learning on multifidelity data, *J. Mach. Learn. Model. Comput.* 3 (1) (2022).

- [37] S. Chakraborty, Transfer learning based multi-fidelity physics informed deep neural network, *J. Comput. Phys.* 426 (2021) 109942.
- [38] Z. Li, S. Zhang, H. Li, K. Tian, Z. Cheng, Y. Chen, B. Wang, On-line transfer learning for multi-fidelity data fusion with ensemble of deep neural networks, *Adv. Eng. Inf.* 53 (2022) 101689.
- [39] S. De, A. Doostan, Neural network training using L1-regularization and bi-fidelity data, *J. Comput. Phys.* 458 (2022) 111010.
- [40] S. Jiang, L.J. Durllofsky, Use of multifidelity training data and transfer learning for efficient construction of subsurface flow surrogate models, *J. Comput. Phys.* 474 (2023) 111800.
- [41] S. De, J. Britton, M. Reynolds, R. Skinner, K. Jansen, A. Doostan, On transfer learning of neural networks using bi-fidelity data for uncertainty propagation, *Int. J. Uncertain. Quantif.* 10 (6) (2020) 543–573.
- [42] M. Aliakbari, M. Mahmoudi, P. Vadasz, A. Arzani, Predicting high-fidelity multiphysics data from low-fidelity fluid flow and transport solvers using physics-informed neural networks, *Int. J. Heat Fluid Flow* 96 (2022) 109002.
- [43] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, *Adv. Neural Inf. Process. Syst.* 30 (2017) 1–15.
- [44] B. Mildenhall, P.P. Srinivasan, M. Tancik, J.T. Barron, R. Ramamoorthi, R. Ng, NeRF: representing scenes as neural radiance fields for view synthesis, *Commun. ACM* 65 (1) (2021) 99–106.
- [45] S. Wang, Y. Teng, P. Perdikaris, Understanding and mitigating gradient flow pathologies in physics-informed neural networks, *SIAM J. Sci. Comput.* 43 (5) (2021) A3055–A3081.
- [46] M. Tancik, P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. Barron, R. Ng, Fourier features let networks learn high frequency functions in low dimensional domains, *Adv. Neural Inf. Process. Syst.* 33 (2020) 7537–7547.
- [47] A. Kumar, A. Raghunathan, R. Jones, T. Ma, P. Liang, Fine-tuning can distort pretrained features and underperform out-of-distribution, (2022). *arXiv preprint arXiv:2202.10054*
- [48] S. Wang, H. Wang, P. Perdikaris, Improved architectures and training algorithms for deep operator networks, *J. Sci. Comput.* 92 (2) (2022) 35.
- [49] K. Taira, S.L. Brunton, S.T.M. Dawson, C.W. Rowley, T. Colonius, B.J. McKeon, O.T. Schmidt, S. Gordueyev, V. Theofilis, L.S. Ukeiley, Modal analysis of fluid flows: an overview, *AIAA J.* 55 (12) (2017) 4013–4041.
- [50] C. Canuto, A. Russo, V. van Kemenade, Stabilized spectral methods for the Navier-Stokes equations: residual-free bubbles and preconditioning, *Comput. Methods Appl. Mech. Eng.* 166 (1-2) (1998) 65–83.
- [51] P.R. Spalart, R.D. Moser, M.M. Rogers, Spectral methods for the Navier-Stokes equations with one infinite and two periodic directions, *J. Comput. Phys.* 96 (2) (1991) 297–324.
- [52] A. Tewari, J. Thies, B. Mildenhall, P. Srinivasan, E. Tretschk, W. Yifan, C. Lassner, V. Sitzmann, R. Martin-Brualla, S. Lombardi, et al., Advances in neural rendering, in: *Computer Graphics Forum*, 41, Wiley Online Library, 2022, pp. 703–735.
- [53] J. Zheng, S. Ramasinghe, S. Lucey, Rethinking positional encoding, (2021). *arXiv preprint arXiv:2107.02561*
- [54] N. Benbarka, T. Höfer, A. Zell, et al., Seeing implicit neural representations as Fourier series, in: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2022, pp. 2041–2050.
- [55] B.B. Damodaran, F. Schnitzler, A. Lambert, P. Hellier, Improved positional encoding for implicit neural representation based compact data representation, (2023). *arXiv preprint arXiv:2311.06059*
- [56] L. Lu, X. Meng, Z. Mao, G.E. Karniadakis, DeepXDE: a deep learning library for solving differential equations, *SIAM Rev.* 63 (1) (2021) 208–228.
- [57] PyTorch Team, Automatic mixed precision (AMP) recipe, 2023, (https://pytorch.org/tutorials/recipes/recipes/amp_recipe.html). Accessed: 2025-02-02.